React + Next.js (App Router) — Complete Patterns Reference

> A living reference for your journey. Start with the React patterns, then layer in Next.js as you go.

Table of Contents

Part 1 — React Fundamentals

#	Pattern
1	Three-State Pattern
2	Fetch Pattern (useEffect)
3	Render Guard Pattern
4	Derived State Pattern
5	Handler Naming Pattern
6	Immutable Update Pattern
7	Controlled Input Pattern
8	Conditional Render Pattern
9	Local Storage Pattern
10	Custom Hook Pattern
11	Form Submit Pattern
12	Modal Pattern
13	Debounced Search Pattern
14	Pagination Pattern
15	Cleanup Pattern

Part 2 — Next.js App Router

#	Pattern
16	Server vs Client Components

#	Pattern
17	File Structure Pattern
18	Layout Pattern
19	Server Data Fetch Pattern
20	Fetch Cache Options Pattern
21	Dynamic Route Pattern
22	API Route Pattern
23	API Route with Params Pattern
24	useRouter Pattern
25	usePathname & useSearchParams Pattern
26	Link Component Pattern
27	Image Component Pattern
28	loading.jsx Pattern
29	error.jsx Pattern
30	Metadata Pattern (SEO)
31	Server Action Pattern
32	Auth Middleware Pattern
33	Environment Variable Pattern
34	Tailwind Component Pattern
35	cn() Utility Pattern

Part 3 — Quick Reference

#	Pattern
—	Where does this code live?
—	App Router File Conventions
—	Import Alias Cheat Sheet
—	Daily Practice Checklist

Part 4 — shadcn/ui

#	Pattern
36	Button Pattern
37	Form Pattern (react-hook-form + zod)
38	Dialog (Modal) Pattern
39	Toast Pattern
40	Card Pattern
41	Data Table Pattern
42	Select Pattern
43	Sheet (Sidebar/Drawer) Pattern
44	Skeleton Loading Pattern
45	shadcn + Server Action Form Pattern
46	Loading Spinner Pattern
47	Error Message Pattern
48	Empty State Pattern
—	Full Render Guard in One Place

Part 5 — Theming

#	Pattern
49	How the Theme System Works
50	Tailwind Colour Utilities
51	Dark Mode Setup Pattern
52	Dark Mode Toggle Pattern
53	Custom Brand Colour Pattern
54	Font Pattern

#	Pattern
55	Conditional Dark Mode Class Pattern

Part 6 — Custom Tailwind Components

#	Pattern
56	Container / Page Wrapper
57	Page Header
58	Section
59	Stats Card
60	Avatar with Fallback Initials
61	Badge / Status Pill
62	Divider with Label
63	Responsive Nav Bar
64	Sidebar Layout
65	Search Input

Part 7 — Icons

#	Pattern
66	Icon Size Pattern
67	Icons in Buttons
68	Icons in Nav / Sidebar
69	Common Lucide Icons Cheat Sheet
70	Brand Icons (react-icons)

Part 8 — Fonts

#	Pattern
71	Google Font Pattern

#	Pattern
72	Two-Font Pattern (body + heading)
73	Local Font Pattern
74	Popular Font Pairings
75	Font Size Scale
76	Common Font Mistakes to Avoid

Part 9 — Images

#	Pattern
77	Local Images (in /public)
78	Remote Images (from a URL or CDN)
79	Fill Pattern (responsive images)
80	Priority / Above-the-Fold Pattern
81	User Avatar / Dynamic Image Pattern
82	Placeholder / Blur Pattern
83	Image in a Card Pattern
84	Common Image Mistakes to Avoid

Part 10 — Colours

#	Pattern
85	The 60-30-10 Rule
86	Semantic Colour Usage
87	Colour Palette Combinations
88	Tailwind Colour Shades Cheat Sheet
89	Colour Pairing Rules
90	Status Colour Conventions
91	Gradient Patterns

Part 11 — Spacing

#	Pattern
92	Tailwind Spacing Scale
93	Padding Patterns
94	Gap Patterns
95	Space Between Related vs Unrelated Items
96	Margin vs Padding
97	Responsive Spacing Pattern
98	Common Spacing Patterns You'll Use Every Day
99	Spacing Mistakes to Avoid

Part 12 — Framer Motion

#	Pattern
100	Core Concept — motion components
101	Fade In Pattern
102	variants Pattern (reusable animations)
103	Stagger Pattern (animating lists)
104	AnimatePresence Pattern (exit animations)
105	Hover & Tap Pattern
106	Page Transition Pattern
107	useAnimation Pattern
108	Scroll Animation Pattern

Part 13 — Progressive Web App (PWA)

#	Pattern
109	Setup Pattern
110	Web Manifest Pattern
111	Icons Pattern
112	Offline Fallback Pattern
113	Install Prompt Pattern
114	Mobile-First Tailwind Patterns
115	Bottom Navigation Pattern
116	Pull to Refresh Pattern

Part 14 — The WOW Factor

Topic
Typography tricks — variable fonts, oversized text, outlined text, mixed weights
Colour & visual tricks — noise texture, mesh gradients, glassmorphism, glow, dot grid
Animation tricks — cursor glow, number counter, magnetic button, typewriter, parallax
Image tricks — colour-matched shadow, avatar stack, progressive loading
Interesting libraries — tsparticles, lottie, canvas-confetti, Shiki and more
UX details — optimistic UI, skeletons, micro confirmations, ⌘K palette, haptic feedback
Number & time formatting — compact numbers, currency, relative timestamps
The Details Checklist — typography, colour, spacing, motion, images, states, mobile, polish

---

Patterns to Add As You Learn

These are intentionally left out for now. Add them to this file as you hit them while building.

Part	Topic	Add when you...
Part 15	**Auth** — NextAuth / Clerk, sessions, protected routes	Build your first login page
Part 16	**Database** — Prisma schema, queries, relations	Connect your first database
Part 17	**TanStack Query** — caching, background refetch, mutations	Find useEffect fetching gets messy
Part 18	**Zustand** — global state, stores, persistence	Need state shared across many components
Part 19	**File Uploads** — UploadThing / Cloudinary	Need users to upload images or files
Part 20	**Optimistic UI** — update UI before server responds	Want instant-feeling interactions
Part 21	**Testing** — Jest, React Testing Library	Start shipping things that matter
Part 22	**Deployment** — Vercel, env vars, preview deploys	Push your first project live

---

PART 1 — React Fundamentals

1. The Three-State Pattern

```jsx
// Single data source
const [data, setData] = useState(null);
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);
```

Always together. Never skip one.

When you have multiple data sources, prefix each group with the resource name:

```jsx
```

```
// ✅ Multiple data sources — prefix each group clearly
const [notes, setNotes] = useState(null);
const [notesLoading, setNotesLoading] = useState(true);
const [notesError, setNotesError] = useState(null);

const [user, setUser] = useState(null);
const [userLoading, setUserLoading] = useState(true);
const [userError, setUserError] = useState(null);

const [activities, setActivities] = useState(null);
const [activitiesLoading, setActivitiesLoading] = useState(true);
const [activitiesError, setActivitiesError] = useState(null);
```

When this gets unwieldy, collapse each group into a single state object:

```jsx
// ✅ Cleaner for many sources — one state object per resource
const [notesState, setNotesState] = useState({ data: null, loading: true, error: nul
const [userState, setUserState] = useState({ data: null, loading: true, error: null

// Update like this
setNotesState(prev => ({ ...prev, loading: false, data: response }));
setNotesState(prev => ({ ...prev, loading: false, error: err.message }));
```

Rule of thumb: 1 source = flat vars. 2–3 sources = prefix them. 4+ sources = reach for TanStack Query (Part 14).

---

2. The Fetch Pattern (useEffect)

```jsx
```

```
useEffect(() => {
  async function fetchData() {
    try {
      setIsLoading(true);
      const response = await someApiCall();
      setData(response.data);
      setError(null);
    } catch (err) {
      setError(err.message);
      setData(null);
    } finally {
      setIsLoading(false);
    }
  }
  fetchData();
}, []);
```

try → setData → catch → setError → finally

---

3. The Render Guard Pattern

```jsx
if (isLoading) return <LoadingSpinner />;
if (error) return <ErrorMessage message={error} />;
if (!data) return <EmptyState />;

return <div>{/* actual UI */}</div>;
```

Order matters: loading → error → empty → UI

---

4. The Derived State Pattern

```jsx
// ❌ Don't store what you can calculate
const [completedCount, setCompletedCount] = useState(0);

// ✅ Calculate it
const completedCount = tasks.filter(t => t.completed).length;
```

---

5. The Handler Naming Pattern

```jsx
const handleAddNote = (newNote) => {
  setNotes([...notes, newNote]);
};

<Sidebar onAddNote={handleAddNote} />
```

handle + action = function. on + action = prop.

---

6. The Immutable Update Pattern

```jsx
// ✅ Add
setNotes([...notes, newNote]);

// ✅ Delete
setNotes(notes.filter(n => n.id !== id));

// ✅ Update
setNotes(notes.map(n => n.id === id ? { ...n, completed: true } : n));

// ✅ Insert at index
setNotes([...notes.slice(0, index), newNote, ...notes.slice(index)]);
```

Never mutate. Always spread, filter, or map.

---

7. The Controlled Input Pattern

```jsx
const [title, setTitle] = useState('');

<input
  value={title}
  onChange={(e) => setTitle(e.target.value)}
/>
```

---

8. The Conditional Render Pattern

```jsx
{isLoading && <Spinner />}

{isLoading ? <Spinner /> : <Content />}

{(() => {
  if (isLoading) return <Spinner />;
  if (error) return <Error />;
  return <Content />;
})()}
```

&& for one condition. ? : for two. IIFE for many.

---

9. The Local Storage Pattern

```jsx
const [notes, setNotes] = useState(() => {
  const saved = localStorage.getItem('notes');
  return saved ? JSON.parse(saved) : [];
});

useEffect(() => {
  localStorage.setItem('notes', JSON.stringify(notes));
}, [notes]);
```

Lazy state for load. useEffect for save.

---

10. The Custom Hook Pattern

```jsx
```

```
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

const [notes, setNotes] = useLocalStorage('notes', []);
```

Extract repeated logic when you use it twice.

---

11. The Form Submit Pattern

```jsx
const [isSubmitting, setIsSubmitting] = useState(false);

async function handleSubmit(e) {
  e.preventDefault();
  setIsSubmitting(true);

  try {
    await saveData(formData);
  } catch (error) {
    setSubmitError(error.message);
  } finally {
    setIsSubmitting(false);
  }
}

<form onSubmit={handleSubmit}>
  <button disabled={isSubmitting}>
    {isSubmitting ? 'Saving...' : 'Save'}
  </button>
</form>
```

---

12. The Modal Pattern

```jsx
function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;

  return (
    <div className="fixed inset-0 bg-black/50" onClick={onClose}>
      <div className="bg-white rounded-lg p-6" onClick={(e) => e.stopPropagation()}>
        {children}
        <button onClick={onClose}>Close</button>
      </div>
    </div>
  );
}
```

Close on overlay click. Stop propagation on modal content.

---

13. The Debounced Search Pattern

```jsx
const [searchQuery, setSearchQuery] = useState('');
const [debouncedQuery, setDebouncedQuery] = useState('');

useEffect(() => {
  const timer = setTimeout(() => setDebouncedQuery(searchQuery), 500);
  return () => clearTimeout(timer);
}, [searchQuery]);

useEffect(() => {
  if (debouncedQuery) fetchSearchResults(debouncedQuery);
}, [debouncedQuery]);
```

---

14. The Pagination Pattern

```jsx
```

```
const [currentPage, setCurrentPage] = useState(1);
const [totalPages, setTotalPages] = useState(1);

useEffect(() => {
  fetchPage(currentPage);
}, [currentPage]);

<button onClick={() => setCurrentPage(p => p - 1)} disabled={currentPage === 1}>
  Previous
</button>
<span>Page {currentPage} of {totalPages}</span>
<button onClick={() => setCurrentPage(p => p + 1)} disabled={currentPage === totalPa
  Next
</button>
```

---

15. The Cleanup Pattern

jsx

```jsx
useEffect(() => {
  let isMounted = true;

  async function fetchData() {
    const response = await apiCall();
    if (isMounted) setData(response.data);
  }

  fetchData();
  return () => { isMounted = false; };
}, []);
```

PART 2 — Next.js App Router

---

16. Server vs Client Components

```
App Router default = Server Component (runs on the server, no hooks, no browser
APIs)
```

```
Add 'use client' at the top = Client Component (runs in the browser, can use
hooks)
```

```jsx
// ServerComponent.jsx — no 'use client', fetches data directly
export default async function UserProfile({ userId }) {
  const user = await db.user.findUnique({ where: { id: userId } });

  return <div>{user.name}</div>;
}
```

```jsx
// ClientComponent.jsx — needs interactivity
'use client';

export default function LikeButton() {
  const [liked, setLiked] = useState(false);
  return <button onClick={() => setLiked(!liked)}>{liked ? '❤️' : '🤍'}</button>;
}
```

Rule of thumb: keep as much as possible as Server Components. Push `'use client'` down to the leaves.

---

17. The File Structure Pattern (App Router)

```
app/
├── layout.jsx          ← root layout (always server component)
├── page.jsx            ← homepage  /
├── loading.jsx         ← auto shown while page loads
├── error.jsx           ← auto shown on error ('use client' required)
├── not-found.jsx       ← auto shown for 404s
|
├── dashboard/
|   ├── layout.jsx      ← nested layout for /dashboard/*
|   ├── page.jsx        ← /dashboard
|   └── settings/
|       └── page.jsx    ← /dashboard/settings
|
└── api/
    └── users/
        └── route.js    ← API route at /api/users
```

Every folder = a route. page.jsx makes it publicly accessible.

---

18. The Layout Pattern

```jsx
// app/layout.jsx
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body className="bg-gray-50 text-gray-900">
        <nav>...</nav>
        <main>{children}</main>
        <footer>...</footer>
      </body>
    </html>
  );
}
```

Layouts persist across route changes — nav and footer don't re-render.

---

19. The Server Data Fetch Pattern

```jsx
// app/posts/page.jsx — Server Component, fetch directly, no useEffect needed
export default async function PostsPage() {
  const posts = await fetch('https://api.example.com/posts', {
    next: { revalidate: 60 }   // re-fetch every 60 seconds (ISR)
  }).then(r => r.json());

  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

No useState. No useEffect. Just async/await at the top level.

---

20. The Fetch Cache Options Pattern

```jsx
// Always fresh (SSR — like getServerSideProps)
fetch(url, { cache: 'no-store' });

// Cached forever (SSG — like getStaticProps)
fetch(url, { cache: 'force-cache' });

// Revalidate every N seconds (ISR)
fetch(url, { next: { revalidate: 60 } });

// Revalidate by tag (on-demand revalidation)
fetch(url, { next: { tags: ['posts'] } });
```

---

21. The Dynamic Route Pattern

```
app/
└── posts/
    └── [id]/
        └── page.jsx    ← /posts/1, /posts/2, /posts/abc
```

```jsx
// app/posts/[id]/page.jsx
export default async function PostPage({ params }) {
  const post = await fetchPost(params.id);

  return <article>{post.title}</article>;
}

// Optional: tell Next.js which pages to pre-build
export async function generateStaticParams() {
  const posts = await fetchAllPosts();
  return posts.map(post => ({ id: String(post.id) }));
}
```

---

22. The API Route Pattern

```jsx
```

```
// app/api/users/route.js

export async function GET(request) {
  const users = await db.user.findMany();
  return Response.json(users);
}

export async function POST(request) {
  const body = await request.json();
  const user = await db.user.create({ data: body });
  return Response.json(user, { status: 201 });
}
```

Each HTTP method = its own exported function. No switch statements.

---

23. The API Route with Params Pattern

```
app/api/users/[id]/route.js  ← /api/users/123
```

```jsx
export async function GET(request, { params }) {
  const user = await db.user.findUnique({ where: { id: params.id } });

  if (!user) {
    return Response.json({ error: 'Not found' }, { status: 404 });
  }

  return Response.json(user);
}

export async function DELETE(request, { params }) {
  await db.user.delete({ where: { id: params.id } });
  return new Response(null, { status: 204 });
}
```

---

24. The useRouter Pattern (Client Navigation)

```jsx
```

```
'use client';
import { useRouter } from 'next/navigation';  // ← NOT 'next/router'

export default function LoginButton() {
  const router = useRouter();

  async function handleLogin() {
    await login();
    router.push('/dashboard');      // navigate
    router.replace('/dashboard');   // navigate without adding to history
    router.back();                  // go back
    router.refresh();               // re-fetch server data for current route
  }

  return <button onClick={handleLogin}>Login</button>;
}
```

---

25. The usePathname & useSearchParams Pattern

jsx

```jsx
'use client';
import { usePathname, useSearchParams } from 'next/navigation';

export default function NavLink({ href, children }) {
  const pathname = usePathname();
  const isActive = pathname === href;

  return (
    <a href={href} className={isActive ? 'font-bold text-blue-600' : 'text-gray-600'
      {children}
    </a>
  );
}
```

jsx

```
// Reading ?page=2&sort=name from the URL
'use client';
import { useSearchParams } from 'next/navigation';

export default function ProductList() {
  const searchParams = useSearchParams();
  const page = searchParams.get('page') ?? '1';
  const sort = searchParams.get('sort') ?? 'name';

  // ...
}
```

---

26. The Link Component Pattern

```jsx
import Link from 'next/link';

// Basic
<Link href="/about">About</Link>

// Dynamic route
<Link href={`/posts/${post.id}`}>Read post</Link>

// With query params
<Link href={{ pathname: '/search', query: { q: 'nextjs' } }}>Search</Link>

// Prefetch on hover (default — just use Link)
// Disable prefetch:
<Link href="/about" prefetch={false}>About</Link>
```

Always use `<Link>` for internal navigation. Never `<a href>` for internal routes.

---

27. The Image Component Pattern

```jsx
```

```
import Image from 'next/image';

// Known dimensions (local or remote with known size)
<Image
  src="/hero.png"
  alt="Hero image"
  width={1200}
  height={600}
  priority          // add for above-the-fold images
/>

// Fill parent container (responsive)
<div className="relative h-64 w-full">
  <Image
    src="/hero.png"
    alt="Hero"
    fill
    className="object-cover"
  />
</div>
```

Always use `<Image>` over `<img>`. It auto-optimises, lazy loads, and prevents layout shift.

---

28. The loading.jsx Pattern

```jsx
// app/dashboard/loading.jsx
// Automatically shown while dashboard/page.jsx is loading

export default function Loading() {
  return (
    <div className="flex items-center justify-center h-64">
      <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-blue-600"
    </div>
  );
}
```

Create this file and Next.js handles the rest. No prop drilling needed.

---

29. The error.jsx Pattern

```jsx
// app/dashboard/error.jsx
'use client';  // ← required

export default function Error({ error, reset }) {
  return (
    <div className="p-6 text-center">
      <h2 className="text-xl font-semibold text-red-600">Something went wrong</h2>
      <p className="text-gray-500 mt-2">{error.message}</p>
      <button
        onClick={reset}
        className="mt-4 px-4 py-2 bg-blue-600 text-white rounded-lg"
      >
        Try again
      </button>
    </div>
  );
}
```

`reset` re-renders the segment. Use it to let the user retry without a full reload.

---

30. The Metadata Pattern (SEO)

```jsx
```

```
// Static metadata
export const metadata = {
  title: 'My App',
  description: 'Welcome to my app',
};

// Dynamic metadata
export async function generateMetadata({ params }) {
  const post = await fetchPost(params.id);

  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      title: post.title,
      images: [post.coverImage],
    },
  };
}
```

Export `metadata` from any page.jsx or layout.jsx. Next.js injects it into `<head>` automatically.

---

31. The Server Action Pattern (Forms without API routes)

jsx

```jsx
// app/actions.js
'use server';

export async function createPost(formData) {
  const title = formData.get('title');
  const body = formData.get('body');

  await db.post.create({ data: { title, body } });
  revalidatePath('/posts');   // re-fetch /posts page data
}
```

jsx

```jsx
// app/posts/new/page.jsx
import { createPost } from '@/app/actions';

export default function NewPostPage() {
  return (
    <form action={createPost}>
      <input name="title" placeholder="Title" className="border rounded p-2 w-full"
      <textarea name="body" placeholder="Body" className="border rounded p-2 w-full
      <button type="submit" className="mt-4 bg-blue-600 text-white px-4 py-2 rounded
        Publish
      </button>
    </form>
  );
}
```

No API route needed. The function runs on the server. Works without JavaScript enabled.

---

32. The Auth Middleware Pattern

```jsx
// middleware.js (at the root, next to app/)
import { NextResponse } from 'next/server';

export function middleware(request) {
  const token = request.cookies.get('token')?.value;
  const isAuthPage = request.nextUrl.pathname.startsWith('/login');

  if (!token && !isAuthPage) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  return NextResponse.next();
}

// Which routes to protect
export const config = {
  matcher: ['/dashboard/:path*', '/settings/:path*'],
};
```

Middleware runs before the page renders — perfect for auth redirects.

---

33. The Environment Variable Pattern

```bash
# .env.local (never commit this)
DATABASE_URL=postgresql://...
NEXTAUTH_SECRET=your-secret

# Public variables (exposed to browser — prefix with NEXT_PUBLIC_)
NEXT_PUBLIC_API_URL=https://api.example.com
```

```jsx
// Server-only (in Server Components, API routes, Server Actions)
const db = new PrismaClient({
  datasources: { db: { url: process.env.DATABASE_URL } }
});

// Client-safe (in Client Components)
const apiUrl = process.env.NEXT_PUBLIC_API_URL;
```

No `NEXT_PUBLIC_` prefix = server only. Always. Secrets never go near the client.

---

34. The Tailwind Component Pattern

```jsx
```

```jsx
// ✅ Extract repeated class strings into variables
const buttonBase = 'px-4 py-2 rounded-lg font-medium transition-colors';
const variants = {
  primary: 'bg-blue-600 text-white hover:bg-blue-700',
  secondary: 'bg-gray-100 text-gray-800 hover:bg-gray-200',
  danger: 'bg-red-600 text-white hover:bg-red-700',
};

export default function Button({ variant = 'primary', children, ...props }) {
  return (
    <button className={`${buttonBase} ${variants[variant]}`} {...props}>
      {children}
    </button>
  );
}

// Usage
<Button variant="primary">Save</Button>
<Button variant="danger">Delete</Button>
```

---

35. The cn() Utility Pattern (Conditional Tailwind Classes)

bash

```bash
npm install clsx tailwind-merge
```

jsx

```jsx
// lib/utils.js
import { clsx } from 'clsx';
import { twMerge } from 'tailwind-merge';

export function cn(...inputs) {
  return twMerge(clsx(inputs));
}
```

jsx

```
// Usage — merges classes and resolves conflicts
import { cn } from '@/lib/utils';

<div className={cn(
  'px-4 py-2 rounded-lg',
  isActive && 'bg-blue-600 text-white',
  isDisabled && 'opacity-50 cursor-not-allowed',
  className   // allow parent overrides
)} />
```

`cn()` is the single most reused utility in any Next.js + Tailwind project.

PART 3 — Quick Reference

The Mental Model: Where does this code live?

Question	Answer
Does it need `useState` / `useEffect`?	Client Component (`'use client'`)
Does it fetch data?	Server Component (async function, no `'use client'`)
Does it handle HTTP requests?	API Route (`app/api/.../route.js`)
Is it a form mutation?	Server Action (`'use server'`)
Does it run before every request?	Middleware (`middleware.js`)

The App Router File Conventions

File	Purpose
`page.jsx`	The UI for a route
`layout.jsx`	Wraps page, persists across navigation
`loading.jsx`	Shown while page loads (Suspense boundary)
`error.jsx`	Shown when page throws (`'use client'` required)
`not-found.jsx`	Shown for 404s
`route.js`	API endpoint (no UI)

File	Purpose
`middleware.js`	Runs before every request (root level)

The Import Alias Cheat Sheet

```jsx
// In next.config.js, @ = your project root
// So instead of:
import { cn } from '../../../lib/utils';

// You write:
import { cn } from '@/lib/utils';
```

Daily Practice Checklist

- [ ] Can you explain the difference between a Server and Client Component?
- [ ] Can you write a Server Component that fetches and renders data?
- [ ] Can you create a dynamic route like `/posts/[id]`?
- [ ] Can you write a GET and POST API route handler?
- [ ] Can you protect a route with middleware?
- [ ] Can you use `useRouter` to navigate programmatically?
- [ ] Can you write a `cn()` call with conditional classes?

Master Part 1 first. Layer in Part 2 as you build. Part 3 is your daily sanity check.

PART 4 — shadcn/ui

> shadcn is not a component library you install — it copies components directly into your project under `components/ui/`. You own the code and can edit it freely.

Setup

```bash
```

```
npx shadcn@latest init
npx shadcn@latest add button
npx shadcn@latest add input card dialog form table toast
```

Components land in `components/ui/`. Import from there, not from a package.

---

36. The Button Pattern

```jsx
import { Button } from '@/components/ui/button';

<Button>Save</Button>
<Button variant="outline">Cancel</Button>
<Button variant="destructive">Delete</Button>
<Button variant="ghost">Dismiss</Button>
<Button variant="secondary">Secondary</Button>
<Button size="sm">Small</Button>
<Button size="lg">Large</Button>
<Button disabled>Disabled</Button>

// Loading state
<Button disabled={isSubmitting}>
  {isSubmitting && <Loader2 className="mr-2 h-4 w-4 animate-spin" />}
  {isSubmitting ? 'Saving...' : 'Save'}
</Button>
```

---

37. The Form Pattern (shadcn + react-hook-form + zod)

```bash
npm install react-hook-form zod @hookform/resolvers
npx shadcn@latest add form input label
```

```jsx
```

```
'use client';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import {
  Form, FormControl, FormField, FormItem, FormLabel, FormMessage
} from '@/components/ui/form';
import { Input } from '@/components/ui/input';
import { Button } from '@/components/ui/button';

// 1. Define schema
const formSchema = z.object({
  email: z.string().email('Invalid email'),
  password: z.string().min(8, 'At least 8 characters'),
});

export default function LoginForm() {
  // 2. Create form
  const form = useForm({
    resolver: zodResolver(formSchema),
    defaultValues: { email: '', password: '' },
  });

  // 3. Handle submit — only called if validation passes
  async function onSubmit(values) {
    await login(values.email, values.password);
  }

  return (
    <Form {...form}>
      <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4">
        <FormField
          control={form.control}
          name="email"
          render={({ field }) => (
            <FormItem>
              <FormLabel>Email</FormLabel>
              <FormControl>
                <Input placeholder="you@example.com" {...field} />
              </FormControl>
              <FormMessage />  {/* shows zod error automatically */}
            </FormItem>
          )}
        />
        <FormField
          control={form.control}
```

```
          name="password"
          render={({ field }) => (
            <FormItem>
              <FormLabel>Password</FormLabel>
              <FormControl>
                <Input type="password" {...field} />
              </FormControl>
              <FormMessage />
            </FormItem>
          )}
        />
        <Button type="submit" className="w-full">Login</Button>
      </form>
    </Form>
  );
}
```

This is the pattern you'll use for every form. Memorise it.

---

38. The Dialog (Modal) Pattern

```jsx
```

```jsx
import {
  Dialog, DialogContent, DialogHeader,
  DialogTitle, DialogTrigger, DialogFooter
} from '@/components/ui/dialog';
import { Button } from '@/components/ui/button';

export default function DeleteDialog({ onConfirm }) {
  return (
    <Dialog>
      <DialogTrigger asChild>
        <Button variant="destructive">Delete</Button>
      </DialogTrigger>
      <DialogContent>
        <DialogHeader>
          <DialogTitle>Are you sure?</DialogTitle>
        </DialogHeader>
        <p className="text-sm text-muted-foreground">This action cannot be undone.</
        <DialogFooter>
          <Button variant="outline">Cancel</Button>
          <Button variant="destructive" onClick={onConfirm}>Delete</Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  );
}
```

`asChild` passes the trigger behaviour to your own element instead of wrapping it.

---

39. The Toast Pattern

```bash
npx shadcn@latest add sonner
```

```jsx
```

```jsx
// app/layout.jsx — add once at the root
import { Toaster } from '@/components/ui/sonner';

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        {children}
        <Toaster />
      </body>
    </html>
  );
}
```

```jsx
// Anywhere in your app
import { toast } from 'sonner';

toast.success('Post published!');
toast.error('Something went wrong.');
toast.info('Changes saved.');
toast.promise(savePost(), {
  loading: 'Saving...',
  success: 'Post saved!',
  error: 'Failed to save.',
});
```

---

40. The Card Pattern

```jsx
```

```jsx
import {
  Card, CardContent, CardDescription,
  CardFooter, CardHeader, CardTitle
} from '@/components/ui/card';

<Card>
  <CardHeader>
    <CardTitle>Plan name</CardTitle>
    <CardDescription>What the plan includes</CardDescription>
  </CardHeader>
  <CardContent>
    <p>Main content goes here.</p>
  </CardContent>
  <CardFooter>
    <Button className="w-full">Get started</Button>
  </CardFooter>
</Card>
```

---

41. The Data Table Pattern

```bash
npx shadcn@latest add table
npm install @tanstack/react-table
```

```jsx
```

```
'use client';
import {
  useReactTable, getCoreRowModel, flexRender
} from '@tanstack/react-table';
import {
  Table, TableBody, TableCell, TableHead, TableHeader, TableRow
} from '@/components/ui/table';

const columns = [
  { accessorKey: 'name', header: 'Name' },
  { accessorKey: 'email', header: 'Email' },
  {
    accessorKey: 'status',
    header: 'Status',
    cell: ({ row }) => (
      <span className={row.original.status === 'active' ? 'text-green-600' : 'text-g
        {row.original.status}
      </span>
    ),
  },
];

export default function UsersTable({ data }) {
  const table = useReactTable({ data, columns, getCoreRowModel: getCoreRowModel() })

  return (
    <Table>
      <TableHeader>
        {table.getHeaderGroups().map(group => (
          <TableRow key={group.id}>
            {group.headers.map(header => (
              <TableHead key={header.id}>
                {flexRender(header.column.columnDef.header, header.getContext())}
              </TableHead>
            ))}
          </TableRow>
        ))}
      </TableHeader>
      <TableBody>
        {table.getRowModel().rows.map(row => (
          <TableRow key={row.id}>
            {row.getVisibleCells().map(cell => (
              <TableCell key={cell.id}>
                {flexRender(cell.column.columnDef.cell, cell.getContext())}
              </TableCell>
            ))}
```

```
          </TableRow>
        ))}
      </TableBody>
    </Table>
  );
}
```

---

42. The Select Pattern

```jsx
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue
} from '@/components/ui/select';

<Select onValueChange={(value) => setRole(value)} defaultValue="user">
  <SelectTrigger className="w-48">
    <SelectValue placeholder="Select role" />
  </SelectTrigger>
  <SelectContent>
    <SelectItem value="admin">Admin</SelectItem>
    <SelectItem value="editor">Editor</SelectItem>
    <SelectItem value="user">User</SelectItem>
  </SelectContent>
</Select>
```

---

43. The Sheet (Sidebar/Drawer) Pattern

```jsx
```

```jsx
import {
  Sheet, SheetContent, SheetHeader,
  SheetTitle, SheetTrigger
} from '@/components/ui/sheet';

<Sheet>
  <SheetTrigger asChild>
    <Button variant="outline">Open menu</Button>
  </SheetTrigger>
  <SheetContent side="left">   {/* left | right | top | bottom */}
    <SheetHeader>
      <SheetTitle>Navigation</SheetTitle>
    </SheetHeader>
    <nav className="mt-6 flex flex-col gap-2">
      <a href="/dashboard">Dashboard</a>
      <a href="/settings">Settings</a>
    </nav>
  </SheetContent>
</Sheet>
```

Perfect for mobile nav drawers.

---

44. The Skeleton Loading Pattern

```jsx
import { Skeleton } from '@/components/ui/skeleton';

// Mirror the shape of your real UI
function UserCardSkeleton() {
  return (
    <div className="flex items-center gap-4 p-4">
      <Skeleton className="h-12 w-12 rounded-full" />
      <div className="space-y-2">
        <Skeleton className="h-4 w-48" />
        <Skeleton className="h-4 w-32" />
      </div>
    </div>
  );
}

// Use in loading.jsx or while data is fetching
{isLoading ? <UserCardSkeleton /> : <UserCard user={user} />}
```

Match the skeleton shape to the real component so the layout doesn't jump.

---

45. The shadcn + Server Action Form Pattern

```jsx
// Combining shadcn Form with Next.js Server Actions
'use client';
import { useFormStatus } from 'react-dom';
import { createPost } from '@/app/actions';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';

function SubmitButton() {
  const { pending } = useFormStatus();   // knows if parent form is submitting
  return (
    <Button type="submit" disabled={pending}>
      {pending ? 'Publishing...' : 'Publish'}
    </Button>
  );
}

export default function NewPostForm() {
  return (
    <form action={createPost} className="space-y-4">
      <div>
        <Label htmlFor="title">Title</Label>
        <Input id="title" name="title" required />
      </div>
      <SubmitButton />
    </form>
  );
}
```

`useFormStatus` **must be in a child component of the form — not the same component.**

---

46. The Loading Spinner Pattern (shadcn)

```bash
npm install lucide-react   # already installed if you use shadcn
```

```jsx
// components/ui/loading-spinner.jsx
import { Loader2 } from 'lucide-react';
import { cn } from '@/lib/utils';

export default function LoadingSpinner({ className }) {
  return (
    <div className={cn('flex items-center justify-center py-12', className)}>
      <Loader2 className="h-8 w-8 animate-spin text-muted-foreground" />
    </div>
  );
}
```

```jsx
// Full page spinner
export function LoadingPage() {
  return (
    <div className="flex min-h-screen items-center justify-center">
      <Loader2 className="h-10 w-10 animate-spin text-muted-foreground" />
    </div>
  );
}
```

```jsx
// Usage — always at the top, before rendering
if (isLoading) return <LoadingSpinner />;
```

---

47. The Error Message Pattern (shadcn)

```bash
npx shadcn@latest add alert
```

```jsx
```

```jsx
// components/ui/error-message.jsx
import { AlertCircle } from 'lucide-react';
import { Alert, AlertDescription, AlertTitle } from '@/components/ui/alert';
import { Button } from '@/components/ui/button';

export default function ErrorMessage({ message, onRetry }) {
  return (
    <Alert variant="destructive" className="my-4">
      <AlertCircle className="h-4 w-4" />
      <AlertTitle>Something went wrong</AlertTitle>
      <AlertDescription className="mt-1">
        {message ?? 'An unexpected error occurred. Please try again.'}
      </AlertDescription>
      {onRetry && (
        <Button
          variant="outline"
          size="sm"
          onClick={onRetry}
          className="mt-3"
        >
          Try again
        </Button>
      )}
    </Alert>
  );
}
```

```jsx
// Usage — after loading check, before empty check
if (isLoading) return <LoadingSpinner />;
if (error) return <ErrorMessage message={error} onRetry={refetch} />;
```

---

48. The Empty State Pattern (shadcn)

```jsx
```

```jsx
// components/ui/empty-state.jsx
import { cn } from '@/lib/utils';
import { Button } from '@/components/ui/button';

export default function EmptyState({ icon: Icon, title, description, action, classNa
  return (
    <div className={cn('flex flex-col items-center justify-center py-16 text-center'
      {Icon && (
        <div className="mb-4 rounded-full bg-muted p-4">
          <Icon className="h-8 w-8 text-muted-foreground" />
        </div>
      )}
      <h3 className="text-lg font-medium">{title}</h3>
      {description && (
        <p className="mt-1 text-sm text-muted-foreground max-w-sm">{description}</p>
      )}
      {action && (
        <Button onClick={action.onClick} className="mt-6">
          {action.label}
        </Button>
      )}
    </div>
  );
}
```

jsx

```jsx
// Usage — after loading and error checks
import { FileText } from 'lucide-react';

if (isLoading) return <LoadingSpinner />;
if (error)     return <ErrorMessage message={error} />;
if (!data?.length) return (
  <EmptyState
    icon={FileText}
    title="No posts yet"
    description="Get started by creating your first post."
    action={{ label: 'Create post', onClick: () => router.push('/posts/new') }}
  />
);

return <PostList posts={data} />;
```

The Full Render Guard in One Place

```jsx
// This is your standard page structure — every time, no exceptions
if (isLoading) return <LoadingSpinner />;
if (error)     return <ErrorMessage message={error} onRetry={refetch} />;
if (!data?.length) return (
  <EmptyState
    icon={SomeIcon}
    title="Nothing here yet"
    description="..."
    action={{ label: 'Create', onClick: handleCreate }}
  />
);

return <YourActualUI data={data} />;
```

Make these three components once, reuse them everywhere.

---

shadcn Quick Reference

Component	Install command
Button, Input, Label	`shadcn@latest add button input label`
Form (with validation)	`shadcn@latest add form`
Dialog / Modal	`shadcn@latest add dialog`
Toast notifications	`shadcn@latest add sonner`
Card	`shadcn@latest add card`
Table	`shadcn@latest add table`
Select / Dropdown	`shadcn@latest add select`
Sheet / Drawer	`shadcn@latest add sheet`
Skeleton loader	`shadcn@latest add skeleton`
Dropdown menu	`shadcn@latest add dropdown-menu`
Tabs	`shadcn@latest add tabs`

Component	Install command
Badge	`shadcn@latest add badge`
Avatar	`shadcn@latest add avatar`
Separator	`shadcn@latest add separator`

These 10 shadcn patterns cover 95% of the UI you'll build. The form pattern (37) is the one to master first.

PART 5 — Theming (shadcn + Tailwind)

> shadcn uses CSS variables for all its colours. Tailwind reads those same variables. Change the variables, everything updates — buttons, cards, inputs, all of it.

---

49. How the Theme System Works

shadcn generates CSS variables in `app/globals.css`. Every shadcn component is built using these variables, not hardcoded colours.

```
css
```

```
/* app/globals.css */
@layer base {
  :root {
    --background: 0 0% 100%;        /* white */
    --foreground: 222 47% 11%;      /* near-black text */
    --primary: 222 47% 11%;         /* main action colour */
    --primary-foreground: 0 0% 98%; /* text on primary */
    --muted: 210 40% 96%;           /* subtle backgrounds */
    --muted-foreground: 215 16% 47%;/* secondary text */
    --border: 214 32% 91%;          /* borders */
    --ring: 222 47% 11%;            /* focus rings */
    --radius: 0.5rem;               /* border radius */
    /* ...and more */
  }

  .dark {
    --background: 222 47% 11%;
    --foreground: 0 0% 98%;
    /* dark mode overrides the same variables */
  }
}
```

You never hardcode colours in components. You always use these variables via Tailwind utilities.

---

50. The Tailwind Colour Utilities (from CSS variables)

```jsx
```

```jsx
// Backgrounds
<div className="bg-background" />     // page background
<div className="bg-card" />           // card surface
<div className="bg-muted" />          // subtle grey bg
<div className="bg-primary" />        // brand colour
<div className="bg-destructive" />    // red / danger

// Text
<p className="text-foreground" />         // primary text
<p className="text-muted-foreground" />   // secondary/hint text
<p className="text-primary" />            // brand colour text
<p className="text-destructive" />        // error text

// Borders
<div className="border border-border" />  // standard border
<div className="ring-1 ring-ring" />      // focus ring

// Border radius (uses --radius variable)
<div className="rounded-md" />   // follows your theme radius
```

Use these instead of `text-gray-500` or `bg-white`. They automatically switch in dark mode.

---

51. The Dark Mode Setup Pattern

```bash
npm install next-themes
```

```jsx
```

```jsx
// components/providers/theme-provider.jsx
'use client';
import { ThemeProvider as NextThemesProvider } from 'next-themes';

export function ThemeProvider({ children }) {
  return (
    <NextThemesProvider
      attribute="class"
      defaultTheme="system"
      enableSystem
    >
      {children}
    </NextThemesProvider>
  );
}
```

```jsx
// app/layout.jsx — wrap your app once
import { ThemeProvider } from '@/components/providers/theme-provider';

export default function RootLayout({ children }) {
  return (
    <html lang="en" suppressHydrationWarning>
      <body>
        <ThemeProvider>
          {children}
        </ThemeProvider>
      </body>
    </html>
  );
}
```

`suppressHydrationWarning` on `<html>` is required — next-themes changes the class on mount.

---

52. The Dark Mode Toggle Pattern

```jsx
```

```
'use client';
import { useTheme } from 'next-themes';
import { Sun, Moon, Monitor } from 'lucide-react';
import { Button } from '@/components/ui/button';
import {
  DropdownMenu, DropdownMenuContent,
  DropdownMenuItem, DropdownMenuTrigger
} from '@/components/ui/dropdown-menu';

export default function ThemeToggle() {
  const { setTheme } = useTheme();

  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <Button variant="ghost" size="icon">
          <Sun className="h-4 w-4 rotate-0 scale-100 transition-all dark:-rotate-90
          <Moon className="absolute h-4 w-4 rotate-90 scale-0 transition-all dark:ro
        </Button>
      </DropdownMenuTrigger>
      <DropdownMenuContent align="end">
        <DropdownMenuItem onClick={() => setTheme('light')}>
          <Sun className="mr-2 h-4 w-4" /> Light
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => setTheme('dark')}>
          <Moon className="mr-2 h-4 w-4" /> Dark
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => setTheme('system')}>
          <Monitor className="mr-2 h-4 w-4" /> System
        </DropdownMenuItem>
      </DropdownMenuContent>
    </DropdownMenu>
  );
}
```

---

53. The Custom Brand Colour Pattern

To change the primary colour across your entire app, you only need to update the CSS variables — not touch any component.

```
css
```

```
/* app/globals.css */
:root {
  /* Change these two to rebrand everything */
  --primary: 262 83% 58%;          /* purple */
  --primary-foreground: 0 0% 100%;
}

.dark {
  --primary: 263 70% 70%;          /* lighter purple for dark mode */
  --primary-foreground: 0 0% 100%;
}
```

Use the shadcn Themes page to pick a palette and copy the generated CSS variables directly.

---

54. The Font Pattern (Next.js + Tailwind)

jsx

```jsx
// app/layout.jsx
import { Inter, Geist_Mono } from 'next/font/google';

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-sans',   // maps to Tailwind's font-sans
});

const mono = Geist_Mono({
  subsets: ['latin'],
  variable: '--font-mono',
});

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={`${inter.variable} ${mono.variable}`}>
      <body className="font-sans antialiased">
        {children}
      </body>
    </html>
  );
}
```

js

```
// tailwind.config.js
theme: {
  extend: {
    fontFamily: {
      sans: ['var(--font-sans)'],
      mono: ['var(--font-mono)'],
    },
  },
},
```

Next.js fonts are self-hosted automatically — no layout shift, no external requests.

---

55. The Conditional Dark Mode Class Pattern

```jsx
// When you need different styles per theme
<div className="bg-white dark:bg-zinc-900" />
<p className="text-gray-900 dark:text-gray-100" />
<img
  src="/logo-light.svg"
  className="block dark:hidden"
/>
<img
  src="/logo-dark.svg"
  className="hidden dark:block"
/>
```

Theming Quick Reference

Task	How
Change brand colour	Edit `--primary` in `globals.css`
Add dark mode	Install `next-themes`, wrap with `ThemeProvider`
Use theme-aware text	`text-foreground`, `text-muted-foreground`
Use theme-aware bg	`bg-background`, `bg-card`, `bg-muted`
Use theme-aware border	`border-border`
Override in dark mode	`dark:bg-zinc-900`

Task	How
Pick a full palette	ui.shadcn.com/themes

Change the CSS variables → everything rebrands. That's the whole system.

PART 6 — Custom Tailwind Components

> These aren't in shadcn. Every project ends up writing them from scratch. Build them once, reuse forever.

---

56. Page Wrapper / Container

```jsx
// components/layout/container.jsx
import { cn } from '@/lib/utils';

export default function Container({ children, className }) {
  return (
    <div className={cn('mx-auto w-full max-w-5xl px-4 sm:px-6 lg:px-8', className)}>
      {children}
    </div>
  );
}

// Usage
<Container>
  <h1>Page content</h1>
</Container>

// Narrow (forms, articles)
<Container className="max-w-2xl">
  <LoginForm />
</Container>
```

You'll use this on every page. Make it first.

---

57. Page Header

```jsx
// components/layout/page-header.jsx
import { cn } from '@/lib/utils';

export default function PageHeader({ title, description, action, className }) {
  return (
    <div className={cn('flex items-start justify-between gap-4 mb-8', className)}>
      <div>
        <h1 className="text-2xl font-semibold tracking-tight">{title}</h1>
        {description && (
          <p className="text-sm text-muted-foreground mt-1">{description}</p>
        )}
      </div>
      {action && <div className="shrink-0">{action}</div>}
    </div>
  );
}

// Usage
<PageHeader
  title="All Posts"
  description="Manage your published content."
  action={<Button>New post</Button>}
/>
```

---

58. Section

```jsx
```

```jsx
// components/layout/section.jsx
import { cn } from '@/lib/utils';

export default function Section({ title, description, children, className }) {
  return (
    <section className={cn('py-8', className)}>
      {(title || description) && (
        <div className="mb-6">
          {title && <h2 className="text-lg font-medium">{title}</h2>}
          {description && <p className="text-sm text-muted-foreground mt-1">{descrip
        </div>
      )}
      {children}
    </section>
  );
}
```

---

59. Stats Card

jsx

```jsx
// components/ui/stat-card.jsx
import { Card, CardContent } from '@/components/ui/card';
import { cn } from '@/lib/utils';

export default function StatCard({ label, value, delta, icon: Icon, className }) {
  const isPositive = delta > 0;

  return (
    <Card className={cn('', className)}>
      <CardContent className="pt-6">
        <div className="flex items-center justify-between">
          <p className="text-sm text-muted-foreground">{label}</p>
          {Icon && <Icon className="h-4 w-4 text-muted-foreground" />}
        </div>
        <p className="text-3xl font-semibold mt-2">{value}</p>
        {delta !== undefined && (
          <p className={cn('text-xs mt-1', isPositive ? 'text-green-600' : 'text-red
            {isPositive ? '+' : ''}{delta}% from last month
          </p>
        )}
      </CardContent>
    </Card>
  );
}

// Usage
<div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
  <StatCard label="Total users" value="2,340" delta={12} icon={Users} />
  <StatCard label="Revenue" value="$12,400" delta={-3} icon={DollarSign} />
  <StatCard label="Active today" value="143" icon={Activity} />
</div>
```

---

60. Avatar with Fallback Initials

jsx

```jsx
// components/ui/user-avatar.jsx
import { Avatar, AvatarFallback, AvatarImage } from '@/components/ui/avatar';

function getInitials(name) {
  return name
    .split(' ')
    .map(n => n[0])
    .slice(0, 2)
    .join('')
    .toUpperCase();
}

export default function UserAvatar({ name, imageUrl, className }) {
  return (
    <Avatar className={className}>
      <AvatarImage src={imageUrl} alt={name} />
      <AvatarFallback className="bg-primary text-primary-foreground text-sm">
        {getInitials(name)}
      </AvatarFallback>
    </Avatar>
  );
}

// Usage
<UserAvatar name="Sarah Connor" imageUrl={user.avatarUrl} />
```

---

61. Badge / Status Pill

jsx

```jsx
// components/ui/status-badge.jsx
import { cn } from '@/lib/utils';

const variants = {
  active:   'bg-green-100 text-green-800 dark:bg-green-900/30 dark:text-green-400',
  inactive: 'bg-gray-100 text-gray-600 dark:bg-gray-800 dark:text-gray-400',
  pending:  'bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yellow-40
  error:    'bg-red-100 text-red-700 dark:bg-red-900/30 dark:text-red-400',
};

export default function StatusBadge({ status, label, className }) {
  return (
    <span className={cn(
      'inline-flex items-center rounded-full px-2.5 py-0.5 text-xs font-medium',
      variants[status] ?? variants.inactive,
      className
    )}>
      {label ?? status}
    </span>
  );
}

// Usage
<StatusBadge status="active" />
<StatusBadge status="pending" label="Under review" />
```

---

62. Divider with Label

jsx

```jsx
// components/ui/divider.jsx
import { cn } from '@/lib/utils';

export default function Divider({ label, className }) {
  if (!label) {
    return <hr className={cn('border-border my-6', className)} />;
  }

  return (
    <div className={cn('relative my-6', className)}>
      <div className="absolute inset-0 flex items-center">
        <span className="w-full border-t border-border" />
      </div>
      <div className="relative flex justify-center">
        <span className="bg-background px-3 text-xs text-muted-foreground">{label}</
      </div>
    </div>
  );
}

// Usage — common on auth pages
<Divider label="or continue with" />
```

---

63. Responsive Nav Bar

jsx

```jsx
// components/layout/navbar.jsx
'use client';
import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { cn } from '@/lib/utils';
import { Button } from '@/components/ui/button';
import ThemeToggle from '@/components/ui/theme-toggle';

const links = [
  { href: '/dashboard', label: 'Dashboard' },
  { href: '/posts', label: 'Posts' },
  { href: '/settings', label: 'Settings' },
];

export default function Navbar() {
  const pathname = usePathname();

  return (
    <header className="sticky top-0 z-50 w-full border-b border-border bg-background
      <div className="mx-auto flex h-14 max-w-5xl items-center justify-between px-4"
        <Link href="/" className="font-semibold">MyApp</Link>

        <nav className="hidden md:flex items-center gap-1">
          {links.map(link => (
            <Link
              key={link.href}
              href={link.href}
              className={cn(
                'px-3 py-1.5 rounded-md text-sm transition-colors',
                pathname === link.href
                  ? 'bg-muted font-medium text-foreground'
                  : 'text-muted-foreground hover:text-foreground hover:bg-muted'
              )}
            >
              {link.label}
            </Link>
          ))}
        </nav>

        <div className="flex items-center gap-2">
          <ThemeToggle />
          <Button size="sm">Sign in</Button>
        </div>
      </div>
    </header>
```

```
  );
}
```

---

64. Sidebar Layout

```jsx
```

```
// components/layout/sidebar-layout.jsx
import { cn } from '@/lib/utils';

const sidebarLinks = [
  { href: '/dashboard', label: 'Dashboard', icon: LayoutDashboard },
  { href: '/posts', label: 'Posts', icon: FileText },
  { href: '/settings', label: 'Settings', icon: Settings },
];

export default function SidebarLayout({ children }) {
  return (
    <div className="flex min-h-screen">
      {/* Sidebar */}
      <aside className="hidden md:flex w-60 shrink-0 flex-col border-r border-border
        <div className="mb-6 px-2 font-semibold">MyApp</div>
        <nav className="flex flex-col gap-1">
          {sidebarLinks.map(({ href, label, icon: Icon }) => (
            <SidebarLink key={href} href={href} icon={Icon}>{label}</SidebarLink>
          ))}
        </nav>
      </aside>

      {/* Main content */}
      <main className="flex-1 overflow-y-auto p-6">
        {children}
      </main>
    </div>
  );
}

// Sidebar link with active state
'use client';
function SidebarLink({ href, icon: Icon, children }) {
  const pathname = usePathname();
  const isActive = pathname === href;

  return (
    <Link
      href={href}
      className={cn(
        'flex items-center gap-3 rounded-md px-3 py-2 text-sm transition-colors',
        isActive
          ? 'bg-primary text-primary-foreground'
          : 'text-muted-foreground hover:bg-muted hover:text-foreground'
      )}
    >
```

```
      <Icon className="h-4 w-4 shrink-0" />
      {children}
    </Link>
  );
}
```

---

65. Search Input

```jsx
```

```jsx
// components/ui/search-input.jsx
'use client';
import { Search, X } from 'lucide-react';
import { Input } from '@/components/ui/input';
import { Button } from '@/components/ui/button';
import { cn } from '@/lib/utils';

export default function SearchInput({ value, onChange, placeholder = 'Search...', cl
  return (
    <div className={cn('relative', className)}>
      <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted
      <Input
        value={value}
        onChange={(e) => onChange(e.target.value)}
        placeholder={placeholder}
        className="pl-9 pr-9"
      />
      {value && (
        <Button
          variant="ghost"
          size="icon"
          onClick={() => onChange('')}
          className="absolute right-1 top-1/2 -translate-y-1/2 h-7 w-7"
        >
          <X className="h-3 w-3" />
        </Button>
      )}
    </div>
  );
}

// Usage
<SearchInput
  value={query}
  onChange={setQuery}
  placeholder="Search posts..."
  className="w-72"
/>
```

Custom Components Quick Reference

Component	What it does
`Container`	Max-width centred wrapper for every page
`PageHeader`	Title + description + optional CTA button
`Section`	Labelled content block within a page
`StatCard`	Metric with label, value, and trend delta
`UserAvatar`	Avatar with auto initials fallback
`StatusBadge`	Coloured pill for active/pending/error states
`Divider`	HR with optional centred label
`Navbar`	Sticky top nav with active link + theme toggle
`SidebarLayout`	Two-column layout with icon nav sidebar
`SearchInput`	Input with search icon and clear button

These aren't fancy — they're just the building blocks every app needs. Write them in your first week and you'll never start from scratch again.

PART 7 — Icons

Which Library to Use

Library	Use when
lucide-react ✅ primary	Always — shadcn uses it, tree-shakeable, consistent style
react-icons	You need a brand logo (GitHub, Twitter, Google etc.)
heroicons	You're already on a Tailwind UI project

```bash
```

```
npm install lucide-react   # already installed with shadcn
npm install react-icons    # only if you need brand icons
```

Rule: use lucide for everything. Add react-icons only for brand logos.

---

66. The Icon Size Pattern

```jsx
import { Settings, User, Bell } from 'lucide-react';

// Standard sizes — stick to these
<Settings className="h-4 w-4" />   // small — inline with text, buttons
<Settings className="h-5 w-5" />   // default — nav, lists
<Settings className="h-6 w-6" />   // medium — headings, cards
<Settings className="h-8 w-8" />   // large — empty states, feature icons
<Settings className="h-10 w-10" /> // xl — hero sections

// Always colour via Tailwind
<Settings className="h-5 w-5 text-muted-foreground" />  // grey
<Settings className="h-5 w-5 text-primary" />           // brand
<Settings className="h-5 w-5 text-destructive" />       // red
<Settings className="h-5 w-5 text-green-500" />         // green
```

---

67. Icons in Buttons

```jsx
```

```jsx
import { Plus, Trash2, Pencil, Loader2, ArrowRight } from 'lucide-react';
import { Button } from '@/components/ui/button';

// Icon + label
<Button><Plus className="mr-2 h-4 w-4" />New post</Button>

// Icon only (use size="icon")
<Button variant="ghost" size="icon">
  <Trash2 className="h-4 w-4" />
</Button>

// Loading spinner (replaces icon while submitting)
<Button disabled={isLoading}>
  {isLoading
    ? <Loader2 className="mr-2 h-4 w-4 animate-spin" />
    : <Plus className="mr-2 h-4 w-4" />
  }
  {isLoading ? 'Creating...' : 'Create'}
</Button>

// Icon on the right (CTAs)
<Button>Get started <ArrowRight className="ml-2 h-4 w-4" /></Button>
```

---

68. Icons in Nav / Sidebar

jsx

```
import {
  LayoutDashboard, FileText, Users,
  Settings, Bell, LogOut, ChevronRight
} from 'lucide-react';

const navLinks = [
  { href: '/dashboard', label: 'Dashboard', icon: LayoutDashboard },
  { href: '/posts',     label: 'Posts',     icon: FileText },
  { href: '/users',     label: 'Users',     icon: Users },
  { href: '/settings',  label: 'Settings',  icon: Settings },
];

{navLinks.map(({ href, label, icon: Icon }) => (
  <Link key={href} href={href} className="flex items-center gap-3 px-3 py-2 rounded-
    <Icon className="h-4 w-4 shrink-0" />
    <span>{label}</span>
  </Link>
))}
```

Always `shrink-0` on nav icons so they don't squish on narrow screens.

---

69. Common Lucide Icons Cheat Sheet

Actions

Icon	Name	Use for
`<Plus />`	Plus	Add, create, new
`<Pencil />`	Pencil	Edit, rename
`<Trash2 />`	Trash2	Delete, remove
`<Search />`	Search	Search input, find
`<X />`	X	Close, dismiss, clear
`<Check />`	Check	Confirm, done, success
`<Copy />`	Copy	Copy to clipboard
`<Download />`	Download	Export, save file
`<Upload />`	Upload	Import, attach

Icon	Name	Use for
`<RefreshCw />`	RefreshCw	Reload, retry, sync
`<Filter />`	Filter	Filter list
`<SlidersHorizontal />`	SlidersHorizontal	Advanced filters
`<MoreHorizontal />`	MoreHorizontal	Row actions menu (…)
`<MoreVertical />`	MoreVertical	Vertical actions menu
`<Send />`	Send	Submit message
`<Share2 />`	Share2	Share
`<ExternalLink />`	ExternalLink	Opens in new tab

Navigation

Icon	Name	Use for
`<ChevronRight />`	ChevronRight	Breadcrumbs, accordions
`<ChevronDown />`	ChevronDown	Dropdowns, expand
`<ChevronLeft />`	ChevronLeft	Back, previous
`<ArrowRight />`	ArrowRight	CTA buttons
`<ArrowLeft />`	ArrowLeft	Back navigation
`<Menu />`	Menu	Mobile hamburger
`<Home />`	Home	Home link

Status & Feedback

Icon	Name	Use for
`<Loader2 />`	Loader2	Loading spinner (use with `animate-spin`)
`<AlertCircle />`	AlertCircle	Error, warning alert
`<CheckCircle2 />`	CheckCircle2	Success message
`<Info />`	Info	Info tooltip/alert

Icon	Name	Use for
`<AlertTriangle />`	AlertTriangle	Warning
`<Ban />`	Ban	Blocked, forbidden

Content & Pages

Icon	Name	Use for
`<FileText />`	FileText	Documents, posts
`<Image />`	Image	Photos, media
`<Folder />`	Folder	Categories, groups
`<Tag />`	Tag	Labels, tags
`<Star />`	Star	Favourites, ratings
`<Heart />`	Heart	Likes
`<MessageSquare />`	MessageSquare	Comments, chat
`<Bell />`	Bell	Notifications
`<Mail />`	Mail	Email
`<Link />`	Link	URL, hyperlink
`<Eye />`	Eye	View, preview
`<EyeOff />`	EyeOff	Hide, password toggle
`<Lock />`	Lock	Private, secure
`<Unlock />`	Unlock	Public, unlocked

Layout & UI

Icon	Name	Use for
`<LayoutDashboard />`	LayoutDashboard	Dashboard nav
`<SidebarOpen />`	SidebarOpen	Toggle sidebar
`<PanelLeft />`	PanelLeft	Sidebar layout

Icon	Name	Use for
`<Table2 />`	Table2	Table view
`<LayoutGrid />`	LayoutGrid	Grid view
`<List />`	List	List view
`<Sun />`	Sun	Light mode
`<Moon />`	Moon	Dark mode
`<Monitor />`	Monitor	System theme

People & Auth

Icon	Name	Use for
`<User />`	User	Profile, account
`<Users />`	Users	Team, members
`<UserPlus />`	UserPlus	Invite, add member
`<LogIn />`	LogIn	Sign in
`<LogOut />`	LogOut	Sign out
`<Shield />`	Shield	Permissions, security
`<KeyRound />`	KeyRound	Password, API keys

Data & Analytics

Icon	Name	Use for
`<BarChart3 />`	BarChart3	Analytics, charts
`<TrendingUp />`	TrendingUp	Growth, positive trend
`<TrendingDown />`	TrendingDown	Decline, negative trend
`<Activity />`	Activity	Live stats, pulse
`<DollarSign />`	DollarSign	Revenue, pricing
`<Calendar />`	Calendar	Dates, scheduling

Icon	Name	Use for
`<Clock />`	Clock	Time, duration
`<Database />`	Database	Data, storage
`<Settings />`	Settings	Config, preferences
`<Cpu />`	Cpu	System, performance

---

70. Brand Icons (react-icons)

For logos shadcn/lucide don't have — social, auth providers, tech stack.

```jsx
import { FaGithub, FaTwitter, FaGoogle, FaApple } from 'react-icons/fa';
import { SiVercel, SiSupabase, SiPrisma, SiTailwindcss } from 'react-icons/si';

// OAuth buttons
<Button variant="outline" className="w-full">
  <FaGithub className="mr-2 h-4 w-4" />
  Continue with GitHub
</Button>

<Button variant="outline" className="w-full">
  <FaGoogle className="mr-2 h-4 w-4" />
  Continue with Google
</Button>

// Social links in footer
<a href="https://twitter.com/..."><FaTwitter className="h-5 w-5" /></a>
<a href="https://github.com/..."><FaGithub className="h-5 w-5" /></a>
```

react-icons prefixes: `Fa` = Font Awesome, `Si` = Simple Icons (brands), `Bi` = BoxIcons, `Hi` = Heroicons.

---

lucide-react for everything. react-icons only when you need a brand logo.

PART 8 — Fonts

> Next.js has a built-in font system (`next/font`) that self-hosts every font automatically. No external requests, no layout shift, no flashing. Always use it over a `<link>` tag in `<head>`.

71. The Google Font Pattern (most common)

```jsx
// app/layout.jsx
import { Inter } from 'next/font/google';

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-sans',  // exposes as a CSS variable
});

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={inter.variable}>
      <body className="font-sans antialiased">
        {children}
      </body>
    </html>
  );
}
```

```js
// tailwind.config.js
theme: {
  extend: {
    fontFamily: {
      sans: ['var(--font-sans)'],
    },
  },
},
```

`antialiased` on `<body>` makes text look sharper on Mac/retina screens. Always include it.

---

72. The Two-Font Pattern (body + heading)

The most common combo: one font for body text, a different one for headings.

```jsx
```

```jsx
// app/layout.jsx
import { Inter, Cal_Sans } from 'next/font/google';

const inter = Inter({
  subsets: ['latin'],
  variable: '--font-sans',
});

const calSans = Cal_Sans({   // or Playfair_Display, Merriweather, etc.
  subsets: ['latin'],
  variable: '--font-heading',
  weight: ['600', '700'],
});

export default function RootLayout({ children }) {
  return (
    <html lang="en" className={`${inter.variable} ${calSans.variable}`}>
      <body className="font-sans antialiased">
        {children}
      </body>
    </html>
  );
}
```

```js
// tailwind.config.js
fontFamily: {
  sans:    ['var(--font-sans)'],
  heading: ['var(--font-heading)'],
},
```

```jsx
// Usage
<h1 className="font-heading text-4xl font-bold">Big heading</h1>
<p className="font-sans text-base">Body text</p>
```

---

73. The Local Font Pattern (custom brand font)

When you have a `.woff2` file from a designer or a paid font.

```jsx
```

```jsx
// app/layout.jsx
import localFont from 'next/font/local';

const brandFont = localFont({
  src: [
    { path: '../public/fonts/brand-regular.woff2', weight: '400' },
    { path: '../public/fonts/brand-medium.woff2',  weight: '500' },
    { path: '../public/fonts/brand-bold.woff2',    weight: '700' },
  ],
  variable: '--font-brand',
});
```

Put font files in `/public/fonts/`. Next.js serves them as static assets.

---

74. Popular Font Pairings

Copy-paste any of these into your layout — they all work with the two-font pattern above.

Vibe	Body	Heading	Used by
Clean SaaS	`Inter`	`Inter`	Linear, Vercel
Editorial	`Inter`	`Playfair_Display`	Medium-style blogs
Modern startup	`Geist`	`Geist`	Vercel (their own font)
Friendly app	`Plus_Jakarta_Sans`	`Plus_Jakarta_Sans`	Common in dashboards
Developer tool	`Inter`	`JetBrains_Mono`	Dev-focused products
Elegant	`Lora`	`Cormorant_Garamond`	Portfolio, luxury brands
Minimal	`DM_Sans`	`DM_Sans`	Clean, modern apps

```jsx
// Example — modern startup (Geist)
import { Geist, Geist_Mono } from 'next/font/google';

const geist = Geist({ subsets: ['latin'], variable: '--font-sans' });
const geistMono = Geist_Mono({ subsets: ['latin'], variable: '--font-mono' });
```

---

75. Font Size Scale (Tailwind)

Stick to this scale — don't invent sizes.

```jsx
// Headings
<h1 className="text-4xl font-semibold tracking-tight">Page title</h1>
<h2 className="text-2xl font-semibold tracking-tight">Section title</h2>
<h3 className="text-xl font-medium">Card title</h3>
<h4 className="text-lg font-medium">Sub section</h4>

// Body
<p className="text-base">Normal body text</p>
<p className="text-sm text-muted-foreground">Secondary / helper text</p>
<p className="text-xs text-muted-foreground">Labels, timestamps, captions</p>

// Special
<code className="font-mono text-sm">inline code</code>
<span className="text-lg font-semibold tabular-nums">$1,240</span>  // numbers
```

`tracking-tight` on large headings. `tabular-nums` on numbers so they don't jump width.

---

76. Common Font Mistakes to Avoid

```jsx
```

```
// ❌ Never load fonts via <link> in layout — causes flash and external request
<head>
  <link href="https://fonts.googleapis.com/css2?family=Inter" rel="stylesheet" />
</head>

// ✅ Always use next/font
import { Inter } from 'next/font/google';

// ❌ Never forget suppressHydrationWarning when using next-themes with fonts
<html className={inter.variable}>  // class changes on mount → hydration mismatch

// ✅ Always add it
<html lang="en" className={inter.variable} suppressHydrationWarning>

// ❌ Applying font variable to <body> instead of <html>
<body className={inter.variable}>  // doesn't reach full document

// ✅ Always on <html>
<html className={inter.variable}>
  <body className="font-sans antialiased">
```

Fonts Quick Reference

Task	How
Load a Google font	`import { Inter } from 'next/font/google'`
Load a local font	`import localFont from 'next/font/local'`
Use two fonts	Two imports, two variables, add both to `<html>`
Make it available in Tailwind	Add `variable` option, extend `fontFamily` in config
Apply to whole app	`className="font-sans antialiased"` on `<body>`
Apply to headings only	`className="font-heading"` on heading elements
Find font names	fonts.google.com — replace spaces with `_` in import

---

One font is usually enough. Two is a deliberate choice. Three is a mistake.

PART 9 — Images

> Next.js has a built-in `<Image>` component that automatically optimises, resizes, lazy loads, and prevents layout shift. Never use a plain `<img>` tag for anything that matters.

---

77. Local Images (in /public)

```jsx
import Image from 'next/image';

// ✅ Import directly — Next.js knows the dimensions automatically
import logo from '@/public/logo.png';

<Image
  src={logo}
  alt="Company logo"
/>

// ✅ Or reference by path — but you must provide width and height
<Image
  src="/hero.png"
  alt="Hero image"
  width={1200}
  height={600}
/>
```

Importing directly is better — no need to specify width/height manually.

---

78. Remote Images (from a URL or CDN)

```jsx
// ✅ Works — but you must whitelist the domain first
<Image
  src="https://images.unsplash.com/photo-abc123"
  alt="A photo"
  width={800}
  height={500}
/>
```

```js
```

```
// next.config.js — whitelist every remote domain you use
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'images.unsplash.com' },
      { protocol: 'https', hostname: 'avatars.githubusercontent.com' },
      { protocol: 'https', hostname: '**.supabase.co' },   // wildcard subdomain
      { protocol: 'https', hostname: '**.cloudinary.com' },
    ],
  },
};

export default nextConfig;
```

You will get an error without this. Add domains as you add image sources.

---

79. The Fill Pattern (responsive images)

Use when you don't know the image dimensions ahead of time, or want it to fill its container.

```jsx
// Parent must be position: relative with a defined size
<div className="relative h-64 w-full">
  <Image
    src="/cover.jpg"
    alt="Cover"
    fill
    className="object-cover"       // crop to fill (like background-size: cover)
  />
</div>

// object-contain — show full image with letterboxing
<div className="relative h-64 w-full">
  <Image
    src="/diagram.png"
    alt="Diagram"
    fill
    className="object-contain"
  />
</div>
```

`fill` without a sized parent = broken layout. Always set `relative` + a height on the wrapper.

---

80. The Priority / Above-the-Fold Pattern

```jsx
// ✅ Add priority to any image visible on first load
// — hero images, logos, above-the-fold banners
<Image
  src="/hero.jpg"
  alt="Hero"
  width={1200}
  height={600}
  priority                // skips lazy loading, fetches immediately
/>

// ❌ Don't add priority to everything — it defeats the purpose
// Only the first 1-2 images on the page need it
```

Without `priority`, the hero image loads late and your Lighthouse score suffers.

---

81. The User Avatar / Dynamic Image Pattern

```jsx
// When src might be null (user hasn't uploaded a photo yet)
import Image from 'next/image';
import { UserAvatar } from '@/components/ui/user-avatar';

function ProfileImage({ user }) {
  if (!user.avatarUrl) {
    return <UserAvatar name={user.name} />;   // fallback to initials
  }

  return (
    <Image
      src={user.avatarUrl}
      alt={user.name}
      width={40}
      height={40}
      className="rounded-full object-cover"
    />
  );
}
```

---

82. The Placeholder / Blur Pattern

Shows a blurred preview while the full image loads — great for slow connections.

```jsx
// For local images — Next.js generates the blur automatically
import heroImage from '@/public/hero.jpg';

<Image
  src={heroImage}
  alt="Hero"
  placeholder="blur"    // blur-up effect while loading
  priority
/>

// For remote images — you must supply a blurDataURL yourself
<Image
  src="https://images.unsplash.com/photo-abc"
  alt="Photo"
  width={800}
  height={500}
  placeholder="blur"
  blurDataURL="..."  // tiny base64 preview
/>
```

For local images `placeholder="blur"` is one prop. For remote you need the base64 too — use blurred.dev to generate it.

---

83. The Image in a Card Pattern

The most common layout — image on top, content below.

```jsx
```

```jsx
import Image from 'next/image';
import { Card, CardContent } from '@/components/ui/card';

function PostCard({ post }) {
  return (
    <Card className="overflow-hidden">   {/* overflow-hidden clips the image corners
      <div className="relative h-48 w-full">
        <Image
          src={post.coverImage}
          alt={post.title}
          fill
          className="object-cover transition-transform duration-300 hover:scale-105"
        />
      </div>
      <CardContent className="pt-4">
        <h3 className="font-medium">{post.title}</h3>
        <p className="text-sm text-muted-foreground mt-1">{post.excerpt}</p>
      </CardContent>
    </Card>
  );
}
```

`overflow-hidden` on the Card is essential — without it the image ignores the rounded corners.

---

84. Common Image Mistakes to Avoid

```jsx
```

```
// ❌ Never use <img> for content images
<img src="/hero.jpg" />

// ✅ Always use Next.js Image
<Image src="/hero.jpg" alt="Hero" width={1200} height={600} />

// ❌ fill without a sized parent
<Image src="/photo.jpg" alt="" fill />

// ✅ fill always needs a relative wrapper with a height
<div className="relative h-64 w-full">
  <Image src="/photo.jpg" alt="" fill className="object-cover" />
</div>

// ❌ Missing alt text — breaks accessibility and Lighthouse
<Image src="/photo.jpg" width={800} height={400} />

// ✅ Always provide meaningful alt text
// For decorative images that screen readers should skip, use alt=""
<Image src="/decoration.png" alt="" width={200} height={200} />

// ❌ Forgetting to whitelist remote domains
<Image src="https://some-cdn.com/photo.jpg" ... />  // throws an error

// ✅ Add the domain to next.config.js remotePatterns first
```

Images Quick Reference

Task	How
Local image (known size)	`<Image src="/file.jpg" width={} height={} alt="" />`
Local image (auto size)	`import img from '@/public/file.jpg'` then `<Image src={img} alt="" />`
Remote image	Whitelist domain in `next.config.js`, then use normally
Fill a container	`fill` prop + `relative` parent with a height + `object-cover`
Above-the-fold image	Add `priority` prop
Blur-up effect	`placeholder="blur"` (auto for local, needs `blurDataURL` for remote)

Task	How
Rounded avatar	`className="rounded-full object-cover"`
Card cover image	Wrap in `relative h-48 w-full`, use `fill` + `object-cover`
Clip to rounded corners	`overflow-hidden` on the parent card/container

Always `<Image>` over `<img>`. Always `alt` text. Always whitelist remote domains. Everything else is just layout.

PART 10 — Colours

> Most beginner projects look off not because of bad layout — but because of bad colour choices. This section gives you a system so you never have to guess.

---

85. The 60-30-10 Rule

The single most useful colour rule in UI design.

```
60% — Neutral (background, surfaces)        bg-background, bg-card, bg-muted
30% — Supporting (text, borders, icons)     text-foreground, border-border
10% — Accent (buttons, links, highlights)   bg-primary, text-primary
```

```jsx
// ✅ Good — accent used sparingly, draws the eye to the one action
<div className="bg-background p-6">                // 60% neutral
  <p className="text-muted-foreground text-sm">   // 30% supporting
    Ready to get started?
  </p>
  <Button className="mt-4">Get started</Button>   // 10% accent (primary)
</div>

// ❌ Bad — too much accent, nothing stands out
<div className="bg-primary p-6">
  <p className="text-primary-foreground">Ready to get started?</p>
  <Button variant="outline">Get started</Button>
</div>
```

If everything is colourful, nothing is. Save your accent colour for the one thing you want clicked.

---

86. Semantic Colour Usage (shadcn)

Always reach for semantic colours before specific ones.

```jsx
// ✅ Semantic — works in light and dark mode automatically
<p className="text-foreground">Primary text</p>
<p className="text-muted-foreground">Secondary text</p>
<p className="text-destructive">Error text</p>
<div className="bg-background">Page background</div>
<div className="bg-card">Card surface</div>
<div className="bg-muted">Subtle background</div>
<div className="bg-primary text-primary-foreground">Brand button</div>
<div className="bg-destructive text-destructive-foreground">Delete button</div>
<div className="border-border">Standard border</div>

// ❌ Avoid hardcoded colours in components — breaks dark mode
<p className="text-gray-600">Secondary text</p>
<div className="bg-white">Card</div>
<div className="border-gray-200">Border</div>
```

---

87. Colour Palette Combinations

Ready-to-use palettes. Each is just a set of CSS variable values for `globals.css`.

Slate (neutral, professional — default shadcn)

```
Primary brand: slate-900 / slate-50 (dark mode)
Good for: SaaS, dashboards, developer tools
```

Zinc (warm neutral — popular alternative)

```
Primary brand: zinc-900 / zinc-100 (dark mode)
Good for: Blogs, portfolios, content sites
```

Blue (trustworthy, corporate)

```
css
```

```
:root { --primary: 221 83% 53%; }   /* blue-500 */
```

```
Good for: Finance, productivity, B2B
Used by: Linear, Notion, Jira
```

Violet (modern, creative)

```
:root { --primary: 262 83% 58%; }   /* violet-600 */
```

```
Good for: AI products, startups, creative tools
Used by: Vercel AI, Loom, many indie SaaS
```

Green (growth, success, health)

```
:root { --primary: 142 71% 45%; }   /* green-500 */
```

```
Good for: Finance (growth), health, sustainability
Used by: Robinhood, Duolingo, Monzo
```

Rose (bold, energetic)

```
:root { --primary: 346 77% 49%; }   /* rose-600 */
```

```
Good for: Food, lifestyle, bold brands
```

Orange (friendly, warm)

```
:root { --primary: 25 95% 53%; }    /* orange-500 */
```

```
Good for: Food delivery, marketplaces, consumer apps
Used by: Airbnb-style, food apps
```

Pick one and commit. Change it by editing `--primary` in `globals.css` only.

---

88. Tailwind Colour Shades Cheat Sheet

Every Tailwind colour goes from `50` (lightest) to `950` (darkest). Here's how to use the scale:

```jsx
// Backgrounds (light shades)
<div className="bg-blue-50" />    // very subtle tint — hover states, tag background
<div className="bg-blue-100" />   // light background — info banners, badges

// Text (dark shades)
<p className="text-blue-700" />   // readable on light bg
<p className="text-blue-900" />   // strong, high contrast

// Borders
<div className="border-blue-200" />   // subtle border
<div className="border-blue-400" />   // visible border

// Interactive elements
<div className="bg-blue-600 hover:bg-blue-700" />  // button — go one shade darker o
<div className="bg-blue-500 hover:bg-blue-600" />  // lighter button

// Dark mode aware badge pattern
<span className="bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-400">
  Badge
</span>
```

The hover rule: always go one shade darker on hover. `bg-blue-600 hover:bg-blue-700`.

---

89. The Colour Pairing Rules

Rules that stop your colour combinations from looking off.

```jsx
```

```
// Rule 1 — Dark text on light bg, light text on dark bg
<div className="bg-blue-600 text-white" />          // ✅
<div className="bg-blue-100 text-blue-900" />       // ✅
<div className="bg-blue-600 text-blue-800" />       // ❌ dark on dark — unreadable

// Rule 2 — When using a colour bg, use that same colour family for text
<span className="bg-green-100 text-green-800" />    // ✅ same family
<span className="bg-green-100 text-blue-800" />     // ❌ mixed families — looks ran

// Rule 3 — Neutral backgrounds, coloured accents only
<div className="bg-background">
  <span className="text-primary font-medium">Active</span>   // ✅ accent on neutral
</div>
<div className="bg-primary">
  <span className="text-yellow-400">Warning</span>           // ❌ accent on accent

// Rule 4 — Never use pure black text — use foreground or near-black
<p className="text-black">Title</p>                // ❌ too harsh
<p className="text-foreground">Title</p>           // ✅ slightly softened
<p className="text-zinc-900">Title</p>             // ✅ near-black, easier on eyes
```

---

90. Status Colour Conventions

These are universal — don't reinvent them.

```jsx
```

```
// What colours mean across all apps
// Green  = success, active, online, positive
// Red    = error, danger, delete, negative
// Yellow = warning, pending, in progress
// Blue   = info, neutral, link
// Gray   = disabled, inactive, muted

// In Tailwind
<span className="text-green-600">Active</span>
<span className="text-red-600">Error</span>
<span className="text-yellow-600">Pending</span>
<span className="text-blue-600">Info</span>
<span className="text-muted-foreground">Disabled</span>

// Dark mode aware (always pair bg + text from same family)
<span className="bg-green-100  text-green-800  dark:bg-green-900/30  dark:text-green
<span className="bg-red-100    text-red-800    dark:bg-red-900/30    dark:text-red-4
<span className="bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yello
<span className="bg-blue-100   text-blue-800   dark:bg-blue-900/30   dark:text-blue-
```

---

91. Gradient Patterns (use sparingly)

```jsx
// Subtle page background gradient
<div className="min-h-screen bg-gradient-to-br from-background to-muted" />

// Hero section gradient
<div className="bg-gradient-to-r from-violet-600 to-indigo-600 text-white px-8 py-16

// Text gradient (eye-catching headings)
<h1 className="bg-gradient-to-r from-violet-600 to-indigo-600 bg-clip-text text-tran
  Build something great
</h1>

// Card with subtle top border accent
<div className="border-t-4 border-t-primary bg-card rounded-lg p-6" />
```

Use gradients for hero sections and headings only. Never on body text or UI controls.

Colours Quick Reference

Rule	Guideline
60-30-10	60% neutral, 30% supporting, 10% accent
Text on coloured bg	Same colour family — `bg-blue-100 text-blue-800`
Hover state	One shade darker — `bg-blue-600 hover:bg-blue-700`
Dark mode	Use semantic classes — `text-foreground`, `bg-card`
Status colours	Green = success, Red = error, Yellow = warning, Blue = info
Change brand colour	Edit `--primary` in `globals.css` only
Gradients	Hero sections and display headings only
Never	Pure black text, mixed colour families, accent on accent

---

Pick one brand colour. Use neutrals for everything else. Break this rule only on purpose.

PART 11 — Spacing

> Spacing is what separates a design that looks "off" from one that looks polished. The good news: you only need a few rules and a consistent scale.

---

92. The Tailwind Spacing Scale

Tailwind's spacing scale is based on `4px` per unit. Stick to multiples of 4.

```
1  =  4px   — hairline gap, icon padding
2  =  8px   — tight gap between related items
3  =  12px  — small padding
4  =  16px  — default padding, base gap
5  =  20px  — slightly generous
6  =  24px  — comfortable padding inside cards
8  =  32px  — section breathing room
10 =  40px  — generous section padding
12 =  48px  — large section gap
16 =  64px  — hero/page-level spacing
20 =  80px  — marketing sections
24 =  96px  — very large hero padding
```

```jsx
// ✅ Stick to the scale — don't invent in-between values
<div className="p-6 gap-4" />    // good
<div className="p-7 gap-3" />    // avoid — off-scale values look inconsistent
```

---

93. Padding Patterns

```jsx
// Page-level padding (container inner padding)
<div className="px-4 sm:px-6 lg:px-8" />      // responsive horizontal padding

// Card padding
<div className="p-4" />    // compact card
<div className="p-6" />    // standard card  ← use this most
<div className="p-8" />    // spacious card, settings panels

// Section padding (vertical breathing room between sections)
<section className="py-8" />    // tight section
<section className="py-12" />   // standard section
<section className="py-16" />   // generous section
<section className="py-24" />   // marketing/hero section

// Button padding (already built into shadcn — don't override)
// sm: px-3 py-1.5 / default: px-4 py-2 / lg: px-6 py-3
```

---

94. Gap Patterns (Flexbox & Grid)

```jsx
// Flex gaps
<div className="flex items-center gap-2" />   // icon + label, tight
<div className="flex items-center gap-3" />   // nav links
<div className="flex flex-col gap-4" />       // form fields
<div className="flex flex-col gap-6" />       // card sections

// Grid gaps
<div className="grid grid-cols-2 gap-4" />   // tight grid
<div className="grid grid-cols-3 gap-6" />   // standard card grid
<div className="grid grid-cols-4 gap-8" />   // spacious grid

// Responsive grid (the pattern you'll use most)
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6" />
```

---

95. The Space Between Related vs Unrelated Items

Spacing communicates relationships. Close = related. Far = separate.

```jsx
```

```jsx
// ✅ Label and input are related — tight gap
<div className="flex flex-col gap-1.5">
  <Label>Email</Label>
  <Input />
</div>

// ✅ Form fields are related — medium gap
<div className="flex flex-col gap-4">
  <FormField name="email" />
  <FormField name="password" />
</div>

// ✅ Form and its submit button are related — slightly more
<div className="flex flex-col gap-6">
  <div className="flex flex-col gap-4">
    <FormField name="email" />
    <FormField name="password" />
  </div>
  <Button type="submit">Login</Button>
</div>

// ✅ Separate sections — large gap
<div className="flex flex-col gap-12">
  <ProfileSection />
  <SecuritySection />
  <DangerZoneSection />
</div>
```

The rule: same group = `gap-2` to `gap-4`. Different groups = `gap-8` to `gap-12`.

---

96. Margin vs Padding — When to Use Which

```jsx
```

```jsx
// Padding — space INSIDE an element (pushes content away from edges)
<div className="p-6">           // inside a card
  <p>Content</p>
</div>

// Margin — space OUTSIDE an element (pushes it away from siblings)
<h2 className="mt-8 mb-4">Section title</h2>

// ✅ Prefer gap over margin for sibling spacing
// gap keeps spacing consistent and doesn't collapse

// ❌ Avoid mixing margin and gap on the same axis
<div className="flex flex-col gap-4">
  <p className="mb-2">Item</p>   // gap + margin = unpredictable
</div>

// ✅ Use margin only for:
// — pushing a single element away from its surroundings
// — mt-auto to push something to the bottom of a flex container
<div className="flex flex-col h-full">
  <nav>...</nav>
  <div className="mt-auto">     // pushes to bottom
    <UserProfile />
  </div>
</div>
```

---

97. Responsive Spacing Pattern

Spacing should increase on larger screens — more room, more breathing space.

```jsx
// Padding grows with screen size
<div className="p-4 md:p-6 lg:p-8" />

// Vertical section padding
<section className="py-12 md:py-16 lg:py-24" />

// Gap grows with screen size
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4 md:gap-6 lg:gap

// Page container (standard pattern — use this on every page)
<main className="mx-auto max-w-5xl px-4 sm:px-6 lg:px-8 py-8 md:py-12" />
```

98. Common Spacing Patterns You'll Use Every Day

jsx

```jsx
// Stack (vertical list of items)
<div className="flex flex-col gap-3">
  {items.map(item => <Item key={item.id} {...item} />)}
</div>

// Inline row (icon + text, badge + label)
<div className="flex items-center gap-2">
  <Icon className="h-4 w-4" />
  <span>Label</span>
</div>

// Split row (left content, right action)
<div className="flex items-center justify-between">
  <h2 className="font-medium">Title</h2>
  <Button size="sm">Action</Button>
</div>

// Card with consistent inner spacing
<div className="rounded-lg border border-border bg-card p-6 flex flex-col gap-4">
  <h3 className="font-medium">Card title</h3>
  <p className="text-sm text-muted-foreground">Description</p>
  <Button>Action</Button>
</div>

// Page layout with header + content
<div className="flex flex-col gap-8">
  <PageHeader title="..." />
  <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
    {/* content */}
  </div>
</div>

// Divider between major sections
<div className="flex flex-col divide-y divide-border">
  <div className="py-4">Section one</div>
  <div className="py-4">Section two</div>
  <div className="py-4">Section three</div>
</div>
```

99. Spacing Mistakes to Avoid

```jsx
// ❌ Arbitrary values — breaks the visual rhythm
<div className="p-[13px] gap-[7px]" />

// ✅ Stick to the scale
<div className="p-3 gap-2" />

// ❌ Too little spacing — cramped, hard to read
<div className="p-1 gap-1">
  <h3>Title</h3>
  <p>Description</p>
  <Button>Action</Button>
</div>

// ✅ Give elements room to breathe
<div className="p-6 gap-4 flex flex-col">
  <h3>Title</h3>
  <p>Description</p>
  <Button>Action</Button>
</div>

// ❌ Same spacing everywhere — no visual hierarchy
<div className="flex flex-col gap-4">
  <h1>Page title</h1>
  <h2>Section title</h2>
  <p>Body text</p>
</div>

// ✅ Bigger gaps between sections, smaller within
<div className="flex flex-col gap-12">
  <section className="flex flex-col gap-4">
    <h2>Section title</h2>
    <p>Body text</p>
  </section>
  <section className="flex flex-col gap-4">
    <h2>Another section</h2>
    <p>Body text</p>
  </section>
</div>
```

Spacing Quick Reference

Context	Class
Icon + label gap	`gap-2`
Form fields	`gap-4`
Card inner padding	`p-6`
Card grid gap	`gap-6`
Section vertical padding	`py-12`
Page horizontal padding	`px-4 sm:px-6 lg:px-8`
Between major sections	`gap-12`
Push to bottom of flex	`mt-auto`
Divider between rows	`divide-y divide-border`
Responsive grid	`grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6`

Consistent spacing is invisible. Inconsistent spacing is all anyone notices.

PART 12 — Framer Motion

> Framer Motion is the animation library for React. It handles the one thing Tailwind can't — smooth, physics-based, state-driven animations. Install it when your first project is working and you want to add life to it.

```bash
npm install framer-motion
```

---

100. The Core Concept — motion components

```jsx
```

```jsx
import { motion } from 'framer-motion';

// Replace any HTML element with its motion equivalent
// <div>   →   <motion.div>
// <p>     →   <motion.p>
// <ul>    →   <motion.ul>
// <button> →  <motion.button>

// Three core props
<motion.div
  initial={{ opacity: 0, y: 20 }}   // start state
  animate={{ opacity: 1, y: 0 }}    // end state
  exit={{ opacity: 0, y: -20 }}     // state when removed from DOM
/>
```

`initial` → `animate` is the animation. `exit` only works inside `<AnimatePresence>`.

---

101. The Fade In Pattern

The most common animation — elements appearing on the page.

```jsx
```

```jsx
import { motion } from 'framer-motion';

// Simple fade in
<motion.div
  initial={{ opacity: 0 }}
  animate={{ opacity: 1 }}
  transition={{ duration: 0.3 }}
>
  Content
</motion.div>

// Fade in + slide up (the go-to entrance animation)
<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  transition={{ duration: 0.4, ease: 'easeOut' }}
>
  Content
</motion.div>

// Fade in + scale up (cards, modals)
<motion.div
  initial={{ opacity: 0, scale: 0.95 }}
  animate={{ opacity: 1, scale: 1 }}
  transition={{ duration: 0.2, ease: 'easeOut' }}
>
  Content
</motion.div>
```

---

102. The variants Pattern (reusable animations)

Instead of repeating `initial`/`animate` on every element, define them once.

```jsx
```

```
import { motion } from 'framer-motion';

const fadeUp = {
  initial: { opacity: 0, y: 20 },
  animate: { opacity: 1, y: 0 },
  transition: { duration: 0.4, ease: 'easeOut' },
};

const fadeIn = {
  initial: { opacity: 0 },
  animate: { opacity: 1 },
  transition: { duration: 0.3 },
};

// Usage — clean and consistent
<motion.h1 variants={fadeUp} initial="initial" animate="animate">
  Page title
</motion.h1>

<motion.p variants={fadeIn} initial="initial" animate="animate">
  Description
</motion.p>
```

---

103. The Stagger Pattern (animating lists)

Children animate in one after another instead of all at once.

```jsx
```

```
import { motion } from 'framer-motion';

const container = {
  hidden: { opacity: 0 },
  show: {
    opacity: 1,
    transition: {
      staggerChildren: 0.1,   // each child delays by 0.1s
    },
  },
};

const item = {
  hidden: { opacity: 0, y: 20 },
  show:   { opacity: 1, y: 0 },
};

// Usage
<motion.ul variants={container} initial="hidden" animate="show">
  {posts.map(post => (
    <motion.li key={post.id} variants={item}>
      <PostCard post={post} />
    </motion.li>
  ))}
</motion.ul>
```

Child components automatically inherit `initial` and `animate` from the parent — you only set it once on the container.

---

104. The AnimatePresence Pattern (exit animations)

For animating elements when they are removed from the DOM — modals, toasts, conditional content.

```jsx
```

```jsx
import { motion, AnimatePresence } from 'framer-motion';

// Wrap conditional content in AnimatePresence
<AnimatePresence>
  {isVisible && (
    <motion.div
      key="modal"                          // key is required
      initial={{ opacity: 0, scale: 0.95 }}
      animate={{ opacity: 1, scale: 1 }}
      exit={{ opacity: 0, scale: 0.95 }}   // plays when isVisible becomes false
      transition={{ duration: 0.2 }}
    >
      <Modal />
    </motion.div>
  )}
</AnimatePresence>

// Animating between two states (tab switching, page transitions)
<AnimatePresence mode="wait">
  <motion.div
    key={activeTab}                    // changing key triggers exit + enter
    initial={{ opacity: 0, x: 20 }}
    animate={{ opacity: 1, x: 0 }}
    exit={{ opacity: 0, x: -20 }}
    transition={{ duration: 0.2 }}
  >
    {tabContent[activeTab]}
  </motion.div>
</AnimatePresence>
```

`key` **is required — it's how Framer Motion knows something is a new element.**

---

105. The Hover & Tap Pattern

```jsx
```

```jsx
import { motion } from 'framer-motion';

// Subtle card hover
<motion.div
  whileHover={{ y: -4, shadow: 'lg' }}
  transition={{ duration: 0.2 }}
  className="rounded-lg border bg-card p-6 cursor-pointer"
>
  <CardContent />
</motion.div>

// Button press effect
<motion.button
  whileHover={{ scale: 1.02 }}
  whileTap={{ scale: 0.97 }}
  transition={{ duration: 0.1 }}
  className="bg-primary text-primary-foreground px-4 py-2 rounded-lg"
>
  Click me
</motion.button>

// Icon button spin on hover
<motion.button
  whileHover={{ rotate: 90 }}
  transition={{ duration: 0.2 }}
>
  <Settings className="h-5 w-5" />
</motion.button>
```

Keep `whileTap` scale slightly less than 1 — it gives a satisfying pressed feeling.

---

106. The Page Transition Pattern (Next.js App Router)

```jsx
```

```jsx
// components/providers/page-transition.jsx
'use client';
import { motion, AnimatePresence } from 'framer-motion';
import { usePathname } from 'next/navigation';

export default function PageTransition({ children }) {
  const pathname = usePathname();

  return (
    <AnimatePresence mode="wait">
      <motion.div
        key={pathname}
        initial={{ opacity: 0, y: 8 }}
        animate={{ opacity: 1, y: 0 }}
        exit={{ opacity: 0, y: -8 }}
        transition={{ duration: 0.25, ease: 'easeInOut' }}
      >
        {children}
      </motion.div>
    </AnimatePresence>
  );
}

// app/layout.jsx
<body>
  <PageTransition>
    {children}
  </PageTransition>
</body>
```

---

107. The useAnimation Pattern (trigger animations programmatically)

```jsx
```

```jsx
'use client';
import { motion, useAnimation } from 'framer-motion';
import { useEffect } from 'react';

export default function ShakeOnError({ hasError, children }) {
  const controls = useAnimation();

  useEffect(() => {
    if (hasError) {
      controls.start({
        x: [0, -8, 8, -8, 8, 0],     // shake keyframes
        transition: { duration: 0.4 },
      });
    }
  }, [hasError]);

  return (
    <motion.div animate={controls}>
      {children}
    </motion.div>
  );
}

// Usage — shakes the form when submission fails
<ShakeOnError hasError={!!submitError}>
  <LoginForm />
</ShakeOnError>
```

---

108. The Scroll Animation Pattern

Animate elements as they scroll into view.

```jsx
```

```
import { motion } from 'framer-motion';

// whileInView — triggers when element enters the viewport
<motion.div
  initial={{ opacity: 0, y: 40 }}
  whileInView={{ opacity: 1, y: 0 }}
  viewport={{ once: true }}          // only animates once, not every time it enters
  transition={{ duration: 0.5, ease: 'easeOut' }}
>
  <FeatureCard />
</motion.div>

// Stagger on scroll — for feature sections, pricing cards
const container = {
  hidden: {},
  show: { transition: { staggerChildren: 0.15 } },
};

const item = {
  hidden: { opacity: 0, y: 30 },
  show:   { opacity: 1, y: 0, transition: { duration: 0.5 } },
};

<motion.div
  variants={container}
  initial="hidden"
  whileInView="show"
  viewport={{ once: true }}
  className="grid grid-cols-3 gap-6"
>
  {features.map(f => (
    <motion.div key={f.id} variants={item}>
      <FeatureCard {...f} />
    </motion.div>
  ))}
</motion.div>
```

`viewport={{ once: true }}` **is important — without it the animation replays every time the user scrolls past.**

Framer Motion Quick Reference

Pattern	When to use
`initial` + `animate`	Any entrance animation
`exit` + `AnimatePresence`	Modals, toasts, conditional content
`variants`	Reusable animation definitions
`staggerChildren`	Animating lists, grids
`whileHover` / `whileTap`	Interactive cards, buttons
`whileInView`	Scroll-triggered animations
`useAnimation`	Programmatic triggers (shake on error, etc.)
`AnimatePresence mode="wait"`	Tab/page switching

Transition ease values

```js
ease: 'easeOut'    // starts fast, slows down — best for entrances
ease: 'easeIn'     // starts slow, speeds up — best for exits
ease: 'easeInOut'  // slow both ends — best for page transitions
ease: 'linear'     // constant speed — good for loaders/spinners
```

Duration guidelines

```
0.1 - 0.2s   micro interactions (hover, tap)
0.2 - 0.3s   small elements appearing (tooltips, dropdowns)
0.3 - 0.5s   page elements, cards, modals
0.5 - 0.8s   page transitions, large sections
```

Start with fade in + slide up. Add stagger to lists. Add hover to cards. That's 90% of what you'll ever need.

PART 13 — Progressive Web App (PWA)

> A PWA makes your Next.js app installable on mobile home screens, work offline, load fast on slow connections, and feel like a native app. If you're targeting mobile, this is not optional.

```bash
npm install next-pwa
```

---

109. The Setup Pattern

```js
// next.config.js
const withPWA = require('next-pwa')({
  dest: 'public',              // output sw.js and workbox files here
  register: true,              // auto register service worker
  skipWaiting: true,           // activate new SW immediately
  disable: process.env.NODE_ENV === 'development',  // don't run in dev
});

module.exports = withPWA({
  // your existing next config here
});
```

```
// .gitignore — add these (next-pwa generates them automatically)
public/sw.js
public/sw.js.map
public/workbox-*.js
public/workbox-*.js.map
```

`disable` in development is important — service workers in dev cause confusing caching issues.

---

110. The Web Manifest Pattern

The manifest tells the browser how your app should look when installed.

```json
```

```
// public/manifest.json
{
  "name": "My App",
  "short_name": "MyApp",
  "description": "What your app does",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#000000",
  "orientation": "portrait",
  "icons": [
    {
      "src": "/icons/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "maskable any"
    },
    {
      "src": "/icons/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "maskable any"
    }
  ]
}
```

jsx

```jsx
// app/layout.jsx — link the manifest
export const metadata = {
  manifest: '/manifest.json',
  themeColor: '#000000',
  appleWebApp: {
    capable: true,
    statusBarStyle: 'default',
    title: 'MyApp',
  },
};
```

`display: standalone` **removes the browser chrome — it looks like a native app when installed.**

---

111. The Icons Pattern

You need two sizes minimum. Use realfavicongenerator.net to generate all sizes from one image.

```
public/
├── icons/
|   ├── icon-192x192.png    ← required (Android home screen)
|   ├── icon-512x512.png    ← required (splash screen)
|   ├── icon-180x180.png    ← Apple touch icon
|   └── favicon.ico         ← browser tab
├── manifest.json
```

```jsx
// app/layout.jsx — link all icon sizes
export const metadata = {
  icons: {
    icon: '/icons/favicon.ico',
    apple: '/icons/icon-180x180.png',
    shortcut: '/icons/icon-192x192.png',
  },
};
```

Use `purpose: "maskable any"` in manifest.json — maskable icons fill the full circle on Android.

---

112. The Offline Fallback Pattern

Show a custom page when the user is offline and tries to navigate somewhere not cached.

```jsx
```

```jsx
// app/offline/page.jsx
export default function OfflinePage() {
  return (
    <div className="flex min-h-screen flex-col items-center justify-center gap-4 tex
      <div className="rounded-full bg-muted p-6">
        <WifiOff className="h-10 w-10 text-muted-foreground" />
      </div>
      <h1 className="text-2xl font-semibold">You're offline</h1>
      <p className="text-muted-foreground max-w-sm">
        Check your connection and try again. Pages you've already visited are still
      </p>
      <Button onClick={() => window.location.reload()}>
        <RefreshCw className="mr-2 h-4 w-4" />
        Try again
      </Button>
    </div>
  );
}
```

```js
// next.config.js — point to the offline fallback
const withPWA = require('next-pwa')({
  dest: 'public',
  register: true,
  skipWaiting: true,
  disable: process.env.NODE_ENV === 'development',
  fallbacks: {
    document: '/offline',   // shown when a page isn't cached
  },
});
```

---

113. The Install Prompt Pattern

Prompt the user to install your app at the right moment — not immediately on first visit.

```jsx
```

```jsx
// hooks/useInstallPrompt.js
'use client';
import { useState, useEffect } from 'react';

export function useInstallPrompt() {
  const [installPrompt, setInstallPrompt] = useState(null);
  const [isInstallable, setIsInstallable] = useState(false);
  const [isInstalled, setIsInstalled] = useState(false);

  useEffect(() => {
    // Check if already installed
    if (window.matchMedia('(display-mode: standalone)').matches) {
      setIsInstalled(true);
      return;
    }

    // Capture the install prompt
    const handler = (e) => {
      e.preventDefault();
      setInstallPrompt(e);
      setIsInstallable(true);
    };

    window.addEventListener('beforeinstallprompt', handler);
    return () => window.removeEventListener('beforeinstallprompt', handler);
  }, []);

  const install = async () => {
    if (!installPrompt) return;
    await installPrompt.prompt();
    const { outcome } = await installPrompt.userChoice;
    if (outcome === 'accepted') setIsInstalled(true);
    setIsInstallable(false);
    setInstallPrompt(null);
  };

  return { isInstallable, isInstalled, install };
}
```

jsx

```jsx
// components/ui/install-banner.jsx
'use client';
import { useInstallPrompt } from '@/hooks/useInstallPrompt';
import { Button } from '@/components/ui/button';
import { Download, X } from 'lucide-react';
import { useState } from 'react';

export default function InstallBanner() {
  const { isInstallable, install } = useInstallPrompt();
  const [dismissed, setDismissed] = useState(false);

  if (!isInstallable || dismissed) return null;

  return (
    <div className="fixed bottom-4 left-4 right-4 z-50 flex items-center justify-bet
      <div>
        <p className="font-medium text-sm">Add to home screen</p>
        <p className="text-xs text-muted-foreground mt-0.5">
          Install for a better experience
        </p>
      </div>
      <div className="flex items-center gap-2 shrink-0">
        <Button size="sm" onClick={install}>
          <Download className="mr-1.5 h-3.5 w-3.5" />
          Install
        </Button>
        <Button
          size="icon"
          variant="ghost"
          className="h-8 w-8"
          onClick={() => setDismissed(true)}
        >
          <X className="h-4 w-4" />
        </Button>
      </div>
    </div>
  );
}

// app/layout.jsx
<body>
  {children}
  <InstallBanner />
</body>
```

Don't show the install prompt immediately. Trigger it after a user action or after they've used the app for a bit.

---

114. Mobile-First Tailwind Patterns

PWA means mobile first. These are the patterns that make it feel native.

```jsx
// Safe area insets — critical for iPhones with notch/home bar
<body className="pb-safe">   {/* respects home bar */}

// In globals.css
@supports (padding-bottom: env(safe-area-inset-bottom)) {
  .pb-safe {
    padding-bottom: env(safe-area-inset-bottom);
  }
}

// Full screen mobile layout
<div className="flex flex-col min-h-[100dvh]">  // dvh = dynamic viewport height (no
  <header className="h-14 shrink-0" />
  <main className="flex-1 overflow-y-auto" />
  <nav className="h-16 shrink-0 pb-safe" />     // bottom nav with safe area
</div>

// Tap target size — minimum 44x44px for touch
<button className="min-h-[44px] min-w-[44px] flex items-center justify-center">
  <Icon className="h-5 w-5" />
</button>

// Prevent text selection on interactive elements
<button className="select-none">Tap me</button>

// Smooth scrolling on iOS
<div className="overflow-y-auto [-webkit-overflow-scrolling:touch]">
```

---

115. The Bottom Navigation Pattern

Mobile apps use bottom nav, not top nav.

```jsx
```

```
// components/layout/bottom-nav.jsx
'use client';
import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { Home, Search, Bell, User } from 'lucide-react';
import { cn } from '@/lib/utils';

const navItems = [
  { href: '/',          label: 'Home',    icon: Home },
  { href: '/search',    label: 'Search',  icon: Search },
  { href: '/activity',  label: 'Activity', icon: Bell },
  { href: '/profile',   label: 'Profile', icon: User },
];

export default function BottomNav() {
  const pathname = usePathname();

  return (
    <nav className="fixed bottom-0 left-0 right-0 z-50 border-t border-border bg-bac
      <div className="flex h-16 items-center justify-around px-2">
        {navItems.map(({ href, label, icon: Icon }) => {
          const isActive = pathname === href;
          return (
            <Link
              key={href}
              href={href}
              className={cn(
                'flex flex-col items-center justify-center gap-1 min-w-[44px] min-h-
                isActive
                  ? 'text-primary'
                  : 'text-muted-foreground hover:text-foreground'
              )}
            >
              <Icon className={cn('h-5 w-5', isActive && 'fill-primary stroke-primar
              <span className="text-[10px] font-medium">{label}</span>
            </Link>
          );
        })}
      </div>
    </nav>
  );
}

// app/layout.jsx
<body>
  <main className="pb-16">   {/* padding so content doesn't hide behind bottom nav *
```

```
    {children}
  </main>
  <BottomNav />
</body>
```

---

116. The Pull to Refresh Pattern

jsx

```
// hooks/usePullToRefresh.js
'use client';
import { useState, useEffect, useRef } from 'react';

export function usePullToRefresh(onRefresh) {
  const [isPulling, setIsPulling] = useState(false);
  const [pullDistance, setPullDistance] = useState(0);
  const startY = useRef(0);
  const threshold = 80;

  useEffect(() => {
    const onTouchStart = (e) => {
      if (window.scrollY === 0) {
        startY.current = e.touches[0].clientY;
      }
    };

    const onTouchMove = (e) => {
      if (!startY.current) return;
      const distance = e.touches[0].clientY - startY.current;
      if (distance > 0 && window.scrollY === 0) {
        setPullDistance(Math.min(distance, threshold * 1.5));
      }
    };

    const onTouchEnd = async () => {
      if (pullDistance >= threshold) {
        setIsPulling(true);
        await onRefresh();
        setIsPulling(false);
      }
      setPullDistance(0);
      startY.current = 0;
    };

    window.addEventListener('touchstart', onTouchStart);
    window.addEventListener('touchmove', onTouchMove);
    window.addEventListener('touchend', onTouchEnd);
    return () => {
      window.removeEventListener('touchstart', onTouchStart);
      window.removeEventListener('touchmove', onTouchMove);
      window.removeEventListener('touchend', onTouchEnd);
    };
  }, [pullDistance, onRefresh]);

  return { isPulling, pullDistance };
```

```
}

// Usage
function FeedPage() {
  const { isPulling, pullDistance } = usePullToRefresh(async () => {
    await refetchFeed();
  });

  return (
    <div>
      {(isPulling || pullDistance > 0) && (
        <div className="flex justify-center py-3">
          <Loader2 className={cn('h-5 w-5', isPulling && 'animate-spin')} />
        </div>
      )}
      <FeedList />
    </div>
  );
}
```

PWA Quick Reference

Task	How
Setup	`npm install next-pwa`, configure `next.config.js`
Manifest	`public/manifest.json` + link in `metadata`
Icons	192×192 and 512×512 minimum, use realfavicongenerator.net
Offline page	`/app/offline/page.jsx` + `fallbacks.document` in config
Install prompt	`beforeinstallprompt` event + custom banner
Safe area (iPhone)	`env(safe-area-inset-bottom)` on bottom nav
Full height (no jump)	`min-h-[100dvh]` instead of `min-h-screen`
Tap target size	`min-h-[44px] min-w-[44px]` on all touch targets
Bottom nav	Fixed bottom, `pb-safe`, `pb-16` on main content
Disable in dev	`disable: process.env.NODE_ENV === 'development'`

A PWA is not a feature you add at the end. Set up the manifest and icons on day one — everything else builds on top.

PART 14 — The WOW Factor

> This section is different from everything else in this file. It's not about patterns — it's about taste. These are the details that separate apps people use from apps people *remember*. You won't use all of these. But knowing they exist means you'll reach for them at the right moment.

Typography Tricks

Variable fonts — one font file, infinite weights. Instead of `font-bold` or `font-light`, you can animate the weight itself.

```
/* Hover to go from thin to bold — feels alive */
.title {
  font-weight: 100;
  transition: font-weight 0.3s ease;
}
.title:hover {
  font-weight: 900;
}
```

Oversized display text — break out of the content width intentionally. Big, confident typography signals quality.

```
<h1 className="text-[clamp(3rem,10vw,9rem)] font-black leading-none tracking-tighter
  Build something
</h1>
```

Mixed font weights in one line — thin and bold together creates tension and hierarchy.

```
<h2 className="text-4xl">
  <span className="font-light">The future of</span>
  <span className="font-black"> everything</span>
</h2>
```

Outlined / stroked text — text that's just an outline, no fill. Striking and modern.

```
css

.outlined {
  -webkit-text-stroke: 2px currentColor;
  color: transparent;
}
```

```
jsx

<h1 className="text-6xl font-black" style={{ WebkitTextStroke: '2px black', color: '
  OUTLINED
</h1>
```

Letter spacing on uppercase labels — all caps with wide tracking looks intentional, not lazy.

```
jsx

<span className="text-xs font-semibold uppercase tracking-widest text-muted-foregrou
  New release
</span>
```

Colour & Visual Tricks

Noise texture overlay — adds grain to flat backgrounds, makes them feel tactile and premium. Used by Linear, Vercel, Stripe.

```
css

/* globals.css */
.noise::after {
  content: '';
  position: fixed;
  inset: 0;
  background-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 256 256' xmlns='http
  pointer-events: none;
  z-index: 9999;
  opacity: 0.15;
}
```

Mesh gradient background — blobs of colour blurred together. The signature look of modern AI/SaaS products.

```jsx
<div className="relative overflow-hidden">
  {/* Colour blobs */}
  <div className="absolute -top-40 -right-40 h-96 w-96 rounded-full bg-violet-400 op
  <div className="absolute -bottom-40 -left-40 h-96 w-96 rounded-full bg-blue-400 op
  <div className="absolute top-1/2 left-1/2 h-64 w-64 -translate-x-1/2 -translate-y-
  {/* Your content on top */}
  <div className="relative z-10">{children}</div>
</div>
```

Glassmorphism — frosted glass effect. Works best over the mesh gradient above.

```jsx
<div className="rounded-2xl border border-white/20 bg-white/10 p-6 backdrop-blur-md"
  Glass card
</div>
```

Colour-shifted dark mode — instead of just going dark grey, shift the hue slightly. Feels more intentional.

```css
.dark {
  --background: 224 71% 4%;    /* dark blue-black instead of pure grey */
  --card: 222 47% 7%;
}
```

Accent glow — a subtle box shadow in your brand colour on hover. Used by Vercel, Linear.

```jsx
<button className="transition-shadow hover:shadow-[0_0_20px_rgba(124,58,237,0.4)]">
  Get started
</button>
```

Subtle grid/dot background pattern — used behind hero sections constantly.

```css
```

```
/* Dot grid */
.dot-grid {
  background-image: radial-gradient(circle, #e2e8f0 1px, transparent 1px);
  background-size: 24px 24px;
}

/* Line grid */
.line-grid {
  background-image:
    linear-gradient(to right, #e2e8f0 1px, transparent 1px),
    linear-gradient(to bottom, #e2e8f0 1px, transparent 1px);
  background-size: 24px 24px;
}
```

Animation & Interaction Tricks

Cursor glow — a radial gradient that follows the cursor. The Linear.app effect everyone tries to copy.

```jsx
```

```
'use client';
import { useEffect, useRef } from 'react';

export default function CursorGlow() {
  const ref = useRef(null);

  useEffect(() => {
    const el = ref.current;
    const move = (e) => {
      const rect = el.getBoundingClientRect();
      const x = e.clientX - rect.left;
      const y = e.clientY - rect.top;
      el.style.setProperty('--x', `${x}px`);
      el.style.setProperty('--y', `${y}px`);
    };
    el.addEventListener('mousemove', move);
    return () => el.removeEventListener('mousemove', move);
  }, []);

  return (
    <div
      ref={ref}
      className="relative overflow-hidden rounded-xl border border-border bg-card p-
      style={{
        background: `radial-gradient(600px circle at var(--x, 50%) var(--y, 50%), rg
      }}
    >
      {/* card content */}
    </div>
  );
}
```

Number counter animation — numbers counting up when they scroll into view. Great for stats sections.

```bash
npm install react-countup
```

```jsx
```

```jsx
import CountUp from 'react-countup';
import { useInView } from 'react-intersection-observer';

function StatNumber({ end, suffix = '' }) {
  const { ref, inView } = useInView({ triggerOnce: true });

  return (
    <span ref={ref}>
      {inView && <CountUp end={end} duration={2} suffix={suffix} />}
    </span>
  );
}

<StatNumber end={10000} suffix="+" />   // counts to 10,000+
<StatNumber end={99} suffix="%" />      // counts to 99%
```

Magnetic button — button that pulls slightly toward the cursor. Subtle and delightful.

```jsx
```

```jsx
'use client';
import { useRef } from 'react';
import { motion, useMotionValue, useSpring } from 'framer-motion';

export default function MagneticButton({ children }) {
  const ref = useRef(null);
  const x = useMotionValue(0);
  const y = useMotionValue(0);
  const springX = useSpring(x, { stiffness: 300, damping: 20 });
  const springY = useSpring(y, { stiffness: 300, damping: 20 });

  const handleMouseMove = (e) => {
    const rect = ref.current.getBoundingClientRect();
    const centerX = rect.left + rect.width / 2;
    const centerY = rect.top + rect.height / 2;
    x.set((e.clientX - centerX) * 0.3);
    y.set((e.clientY - centerY) * 0.3);
  };

  const handleMouseLeave = () => {
    x.set(0);
    y.set(0);
  };

  return (
    <motion.button
      ref={ref}
      style={{ x: springX, y: springY }}
      onMouseMove={handleMouseMove}
      onMouseLeave={handleMouseLeave}
    >
      {children}
    </motion.button>
  );
}
```

Typewriter effect — text that types itself. Great for hero taglines.

```bash
npm install typewriter-effect
```

```jsx
```

```
import Typewriter from 'typewriter-effect';

<h1 className="text-5xl font-bold">
  <Typewriter
    options={{
      strings: ['Build faster.', 'Ship smarter.', 'Look great.'],
      autoStart: true,
      loop: true,
      delay: 50,
      deleteSpeed: 30,
    }}
  />
</h1>
```

Smooth scroll with active section highlight — the nav link lights up as you scroll past each section. Used by every good landing page.

jsx

```
'use client';
import { useEffect, useState } from 'react';

export function useActiveSection(sectionIds) {
  const [activeId, setActiveId] = useState('');

  useEffect(() => {
    const observer = new IntersectionObserver(
      (entries) => {
        entries.forEach((entry) => {
          if (entry.isIntersecting) setActiveId(entry.target.id);
        });
      },
      { rootMargin: '-50% 0px -50% 0px' }  // triggers when section is in middle of
    );

    sectionIds.forEach((id) => {
      const el = document.getElementById(id);
      if (el) observer.observe(el);
    });

    return () => observer.disconnect();
  }, [sectionIds]);

  return activeId;
}
```

Parallax scroll — elements move at different speeds as you scroll. Framer Motion makes this easy.

```jsx
'use client';
import { motion, useScroll, useTransform } from 'framer-motion';
import { useRef } from 'react';

export default function ParallaxSection() {
  const ref = useRef(null);
  const { scrollYProgress } = useScroll({ target: ref });
  const y = useTransform(scrollYProgress, [0, 1], ['0%', '30%']);

  return (
    <div ref={ref} className="relative h-[500px] overflow-hidden">
      <motion.div style={{ y }} className="absolute inset-0">
        <Image src="/hero.jpg" alt="" fill className="object-cover" />
      </motion.div>
      <div className="relative z-10 flex h-full items-center justify-center">
        <h1 className="text-white text-5xl font-bold">Hero Text</h1>
      </div>
    </div>
  );
}
```

Image Tricks

Image with colour-matched shadow — the shadow matches the image colours, not just grey. Used by Apple for product shots.

```jsx
<div className="relative">
  <img
    src={product.image}
    className="rounded-2xl"
    style={{
      filter: 'drop-shadow(0 20px 40px rgba(124, 58, 237, 0.4))'
    }}
  />
</div>
```

Avatar stack — overlapping avatars showing multiple users. Used in collaboration tools.

jsx

```jsx
function AvatarStack({ users, max = 4 }) {
  const visible = users.slice(0, max);
  const remaining = users.length - max;

  return (
    <div className="flex items-center">
      {visible.map((user, i) => (
        <div
          key={user.id}
          className="relative rounded-full ring-2 ring-background"
          style={{ marginLeft: i === 0 ? 0 : '-8px', zIndex: visible.length - i }}
        >
          <UserAvatar name={user.name} imageUrl={user.avatar} className="h-8 w-8" />
        </div>
      ))}
      {remaining > 0 && (
        <div
          className="relative flex h-8 w-8 items-center justify-center rounded-full
          style={{ marginLeft: '-8px' }}
        >
          +{remaining}
        </div>
      )}
    </div>
  );
}
```

Progressive image loading — low quality image loads first, full image fades in on top.

jsx

```
'use client';
import { useState } from 'react';
import Image from 'next/image';
import { cn } from '@/lib/utils';

export default function ProgressiveImage({ src, lowResSrc, alt, ...props }) {
  const [isLoaded, setIsLoaded] = useState(false);

  return (
    <div className="relative">
      {/* Low res placeholder */}
      <Image src={lowResSrc} alt="" {...props} className="blur-sm scale-110" />
      {/* Full res fades in */}
      <Image
        src={src}
        alt={alt}
        {...props}
        className={cn(
          'absolute inset-0 transition-opacity duration-500',
          isLoaded ? 'opacity-100' : 'opacity-0'
        )}
        onLoad={() => setIsLoaded(true)}
      />
    </div>
  );
}
```

Interesting Libraries Worth Knowing

Library	What it does	Wow factor
tsparticles	Particle systems, confetti, snow	Hero backgrounds that move
react-spring	Physics-based animations	More natural than Framer Motion for certain things
lottie-react	Plays After Effects animations in React	Icon animations, loading screens
react-intersection-observer	Detects when elements enter viewport	Scroll-triggered anything
react-countup	Animated number counters	Stats sections
typewriter-effect	Typing animation	Hero taglines

Library	What it does	Wow factor
react-hot-toast	Beautiful toast notifications	Alternative to Sonner
canvas-confetti	Confetti explosion	Success moments, celebrations
react-spring/parallax	Parallax scrolling sections	Immersive landing pages
Shiki	Beautiful code syntax highlighting	Dev tool landing pages

UX Details That Feel Premium

Optimistic UI — update the UI instantly, sync with server in background. Don't make users wait for a loading spinner to like a post.

Skeleton screens over spinners — spinners say "wait". Skeletons say "almost there". Always prefer skeletons for content.

Empty states with personality — don't just say "No items found." Give it an illustration, a helpful message, and a clear action. It shows you thought about the user.

Micro confirmations — small visual feedback for every action. Button briefly changes colour on click. Heart bounces when liked. Checkbox animates when checked.

Keyboard shortcuts — power users notice them. Show them in tooltips. ⌘K for command palette, ? for help modal.

Command palette (⌘K) — the feature that makes apps feel professional.

```bash
npx shadcn@latest add command
```

```jsx
```

```
import {
  CommandDialog, CommandInput, CommandList,
  CommandItem, CommandGroup, CommandEmpty
} from '@/components/ui/command';

// Open with ⌘K
useEffect(() => {
  const down = (e) => {
    if (e.key === 'k' && (e.metaKey || e.ctrlKey)) {
      e.preventDefault();
      setOpen((open) => !open);
    }
  };
  document.addEventListener('keydown', down);
  return () => document.removeEventListener('keydown', down);
}, []);
```

Haptic feedback on mobile — physical feedback on tap. One line, huge native feel difference.

```js
// Trigger haptic feedback on supported devices
const haptic = () => {
  if ('vibrate' in navigator) navigator.vibrate(10);
};

<button onClick={() => { haptic(); handleAction(); }}>
  Tap me
</button>
```

Smooth number formatting — never show raw numbers. Format them properly.

```js
```

```
// utils/format.js
export const formatNumber = (n) =>
  new Intl.NumberFormat('en', { notation: 'compact' }).format(n);
  // 1200 → "1.2K"   1200000 → "1.2M"

export const formatCurrency = (n, currency = 'USD') =>
  new Intl.NumberFormat('en', { style: 'currency', currency }).format(n);
  // 1234.5 → "$1,234.50"

export const formatRelativeTime = (date) => {
  const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
  const diff = (new Date(date) - new Date()) / 1000;
  if (Math.abs(diff) < 60)  return rtf.format(Math.round(diff), 'seconds');
  if (Math.abs(diff) < 3600) return rtf.format(Math.round(diff / 60), 'minutes');
  if (Math.abs(diff) < 86400) return rtf.format(Math.round(diff / 3600), 'hours');
  return rtf.format(Math.round(diff / 86400), 'days');
  // → "2 minutes ago"  "yesterday"  "in 3 days"
};
```

Focus states that look good — the default browser focus ring is ugly. Style it to match your brand.

```
/* globals.css */
:focus-visible {
  outline: 2px solid hsl(var(--primary));
  outline-offset: 2px;
  border-radius: 4px;
}
```

Scroll progress bar — thin line at the top of the page showing how far down you've scrolled. Common on blog posts.

```
```

```
'use client';
import { motion, useScroll, useSpring } from 'framer-motion';

export default function ScrollProgress() {
  const { scrollYProgress } = useScroll();
  const scaleX = useSpring(scrollYProgress, { stiffness: 100, damping: 30 });

  return (
    <motion.div
      style={{ scaleX, transformOrigin: 'left' }}
      className="fixed top-0 left-0 right-0 h-1 bg-primary z-50"
    />
  );
}
```

Smooth page scroll restoration — when a user navigates back, restore their scroll position smoothly.

```js
// next.config.js
const nextConfig = {
  experimental: {
    scrollRestoration: true,
  },
};
```

The Details Checklist

These are the things great apps have that average ones don't. Go through this list before you call anything done.

```
Typography
  [ ] Consistent type scale — not random sizes
  [ ] Proper line height on body text (1.6–1.7)
  [ ] Tight tracking on large headings
  [ ] No more than 2 fonts

Colour
  [ ] Single brand colour used sparingly
  [ ] Semantic colours for status (green/red/yellow)
  [ ] Dark mode tested and working
  [ ] No pure black text (#000) — use near-black

Spacing
  [ ] Consistent scale — multiples of 4
  [ ] More space between sections than within
  [ ] Content doesn't touch screen edges on mobile

Motion
  [ ] Entrances are easeOut
  [ ] Exits are easeIn
  [ ] Nothing takes longer than 0.4s
  [ ] Lists stagger, not all-at-once

Images
  [ ] All images have alt text
  [ ] No layout shift (width/height or fill set)
  [ ] Hero image has priority prop
  [ ] Remote domains whitelisted

States
  [ ] Loading state for every async action
  [ ] Error state with a retry option
  [ ] Empty state with an action
  [ ] Disabled state on buttons while submitting

Mobile
  [ ] Tap targets are 44×44px minimum
  [ ] Bottom nav instead of top nav
  [ ] Safe area padding on iPhone
  [ ] No horizontal scroll

Polish
  [ ] Focus rings are styled
  [ ] Numbers are formatted (1.2K not 1200)
  [ ] Relative timestamps (2 min ago not 2024-01-01)
```

```
  [ ] Haptic feedback on key interactions
  [ ] Keyboard shortcuts for power users
```

Average apps are built. Great apps are finished. The checklist above is the difference.